

Mining Frequent Patterns, Association and Correlations

What Is Frequent Pattern Analysis?

- **Frequent pattern**: a pattern (a set of items, subsequences, substructures, etc.) that occurs frequently in a data set
- First proposed by Agrawal, Imielinski, and Swami [AIS93] in the context of **frequent itemsets** and **association rule mining**
- Motivation: Finding inherent regularities in data
 - What products were often purchased together?— Beer and diapers?!
 - What are the subsequent purchases after buying a PC?
 - What kinds of DNA are sensitive to this new drug?
 - Can we automatically classify web documents?
- Applications
 - Basket data analysis, cross-marketing, catalog design, sale campaign analysis, Web log (click stream) analysis, and DNA sequence analysis.

Why Is Freq. Pattern Mining Important?

- Freq. pattern: An intrinsic and important property of datasets
- Foundation for many essential data mining tasks
 - Association, correlation, and causality analysis
 - Sequential, structural (e.g., sub-graph) patterns
 - Pattern analysis in spatiotemporal, multimedia, time-series, and stream data
 - Classification: discriminative, frequent pattern analysis
 - Cluster analysis: frequent pattern-based clustering
 - Data warehousing: iceberg cube and cube-gradient
 - Semantic data compression: fascicles
 - Broad applications

The model: data

- $I = \{i_1, i_2, \dots, i_m\}$: a set of *items*.
- Transaction t :
 - t a set of items, and $t \subseteq I$.
- Transaction Database T : a set of transactions $T = \{t_1, t_2, \dots, t_n\}$.

Transaction data: supermarket data

- Market basket transactions:
 - t1: {bread, cheese, milk}
 - t2: {apple, eggs, salt, yogurt}
 -
 - tn: {biscuit, eggs, milk}
- Concepts:
 - *An item*: an item/article in a basket
 - *I*: the set of all items sold in the store
 - *A transaction*: items purchased in a basket; it may have TID (transaction ID)
 - *A transactional dataset*: A set of transactions

Transaction data: a set of documents

- **A text document data set. Each document is treated as a “bag” of keywords**

doc1: Student, Teach, School

doc2: Student, School

doc3: Teach, School, City, Game

doc4: Baseball, Basketball

doc5: Basketball, Player, Spectator

doc6: Baseball, Coach, Game, Team

doc7: Basketball, Team, City, Game

The model: rules

- A transaction t contains X , a set of items (itemset) in I , if $X \subseteq t$.
- An association rule is an implication of the form:

$$X \rightarrow Y, \text{ where } X, Y \subset I, \text{ and } X \cap Y = \emptyset$$

- An itemset is a set of items.
 - E.g., $X = \{\text{milk, bread, cereal}\}$ is an itemset.
- A k -itemset is an itemset with k items.
 - E.g., $\{\text{milk, bread, cereal}\}$ is a 3-itemset

Rule strength measures

- **Support:** The rule holds with **support** sup in T (the transaction data set) if $sup\%$ of transactions contain $X \cup Y$.
 - $sup = \Pr(X \cup Y)$.
- **Confidence:** The rule holds in T with **confidence** $conf$ if $conf\%$ of transactions that contain X also contain Y .
 - $conf = \Pr(Y \mid X)$
- An association rule is a pattern that states when X occurs, Y occurs with certain probability.

Support and Confidence

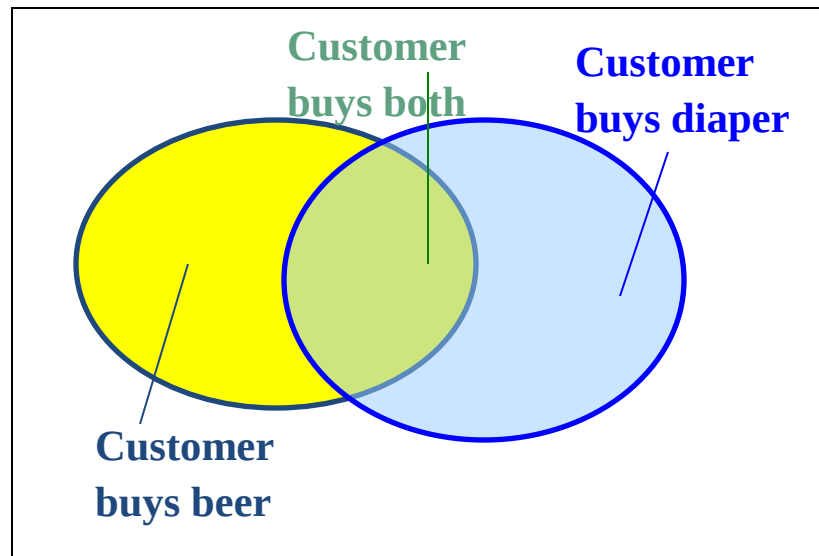
- **Support count**: The support count of an itemset X , denoted by $X.count$, in a data set T is the number of transactions in T that contain X . Assume T has n transactions.

- Then,
$$support = \frac{(X \cup Y).count}{n}$$

$$confidence = \frac{(X \cup Y).count}{X.count}$$

Basic Concepts: Frequent Patterns

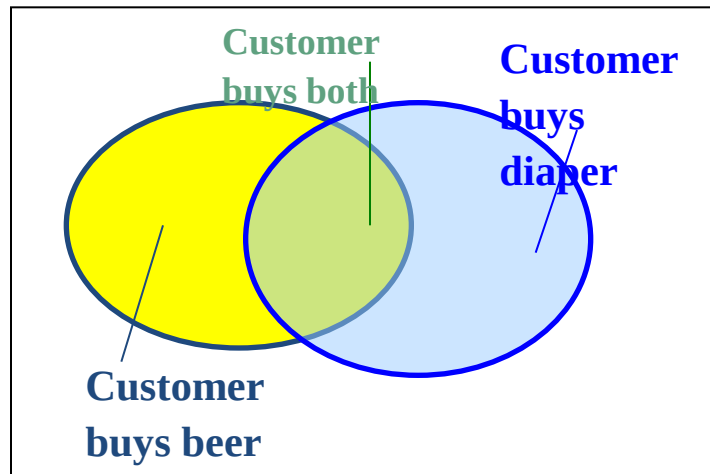
Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- **itemset**: A set of one or more items
- **k-itemset** $X = \{x_1, \dots, x_k\}$
- **(absolute) support**, or, **support count** of X : Frequency or occurrence of an itemset X
- **(relative) support**, s , is the fraction of transactions that contains X (i.e., the **probability** that a transaction contains X)
- An itemset X is **frequent** if X 's support is no less than a *minsup* threshold

Basic Concepts: Association Rules

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- Find all the rules $X \rightarrow Y$ with minimum support and confidence
 - support, s , probability that a transaction contains $X \cup Y$
 - confidence, c , conditional probability that a transaction having X also contains Y

Let $minsup = 50\%$, $minconf = 50\%$

Freq. Pat.: Beer:3, Nuts:3, Diaper:4, Eggs:3, {Beer, Diaper}:3

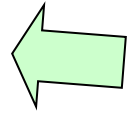
- Association rules: (many more!)
 - $Beer \rightarrow Diaper$ (60%, 100%)
 - $Diaper \rightarrow Beer$ (60%, 75%)

Computational Complexity of Frequent Itemset Mining

- How many itemsets are potentially to be generated in the worst case?
 - The number of frequent itemsets to be generated is sensitive to the minsup threshold
 - When minsup is low, there exist potentially an exponential number of frequent itemsets
 - The worst case: M^N where M : # distinct items, and N : max length of transactions
- The worst case complexity vs. the expected probability
 - Ex. Suppose Walmart has 10^4 kinds of products
 - The chance to pick up one product 10^{-4}
 - The chance to pick up a particular set of 10 products: $\sim 10^{-40}$
 - What is the chance this particular set of 10 products to be frequent 10^3 times in 10^9 transactions?

Scalable Frequent Itemset Mining Methods

- Apriori: A Candidate Generation-and-Test Approach
- Improving the Efficiency of Apriori
- FPGrowth: A Frequent Pattern-Growth Approach
- ECLAT: Frequent Pattern Mining with Vertical Data Format



The Downward Closure Property and Scalable Mining Methods

- The **downward closure** property of frequent patterns
 - Any subset of a frequent itemset must be frequent
 - If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
 - i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}
- Scalable mining methods: Three major approaches
 - Apriori
 - Freq. pattern growth
 - Vertical data format approach

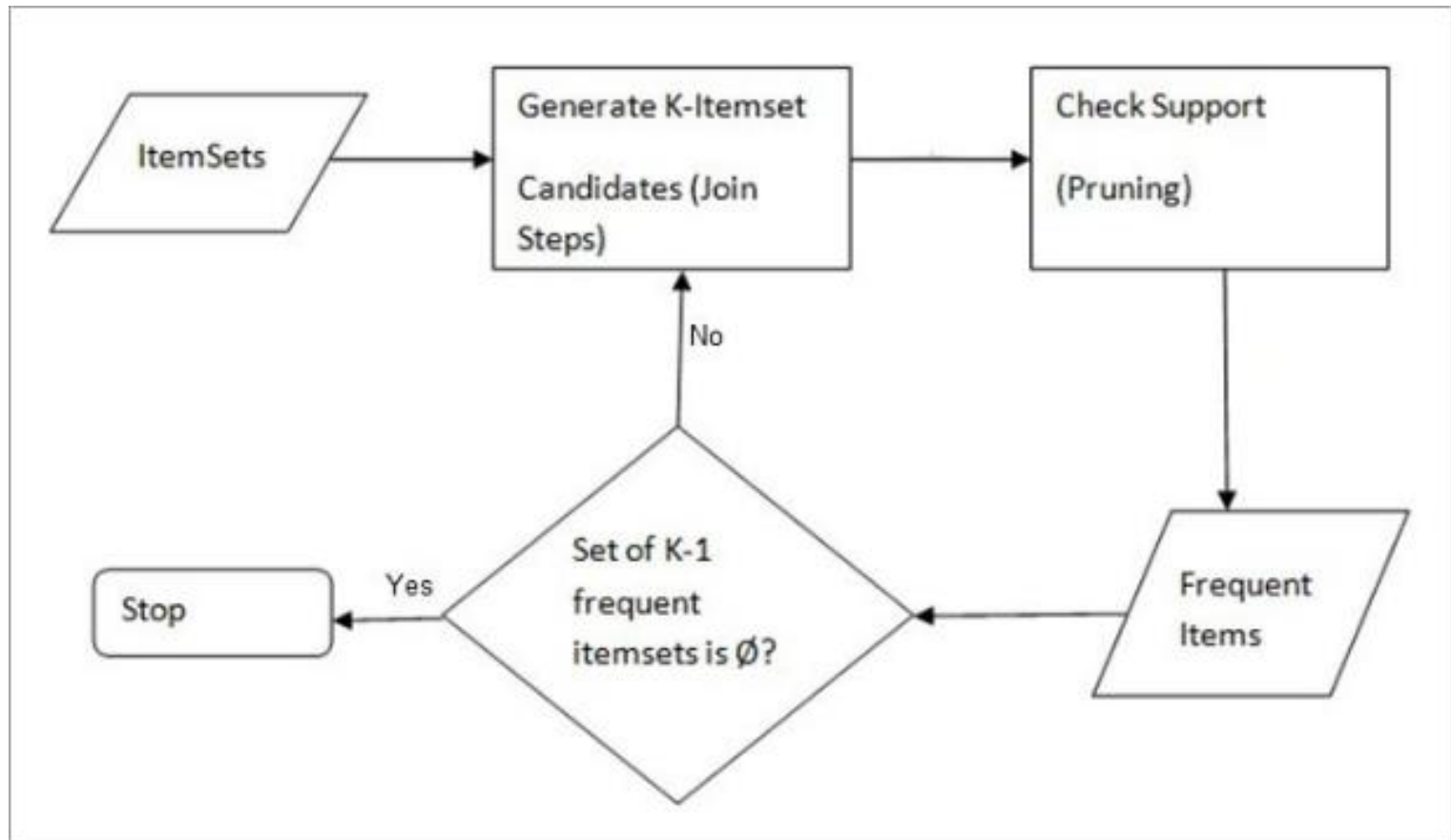
Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is **any** itemset which is infrequent, its superset should not be generated/tested!

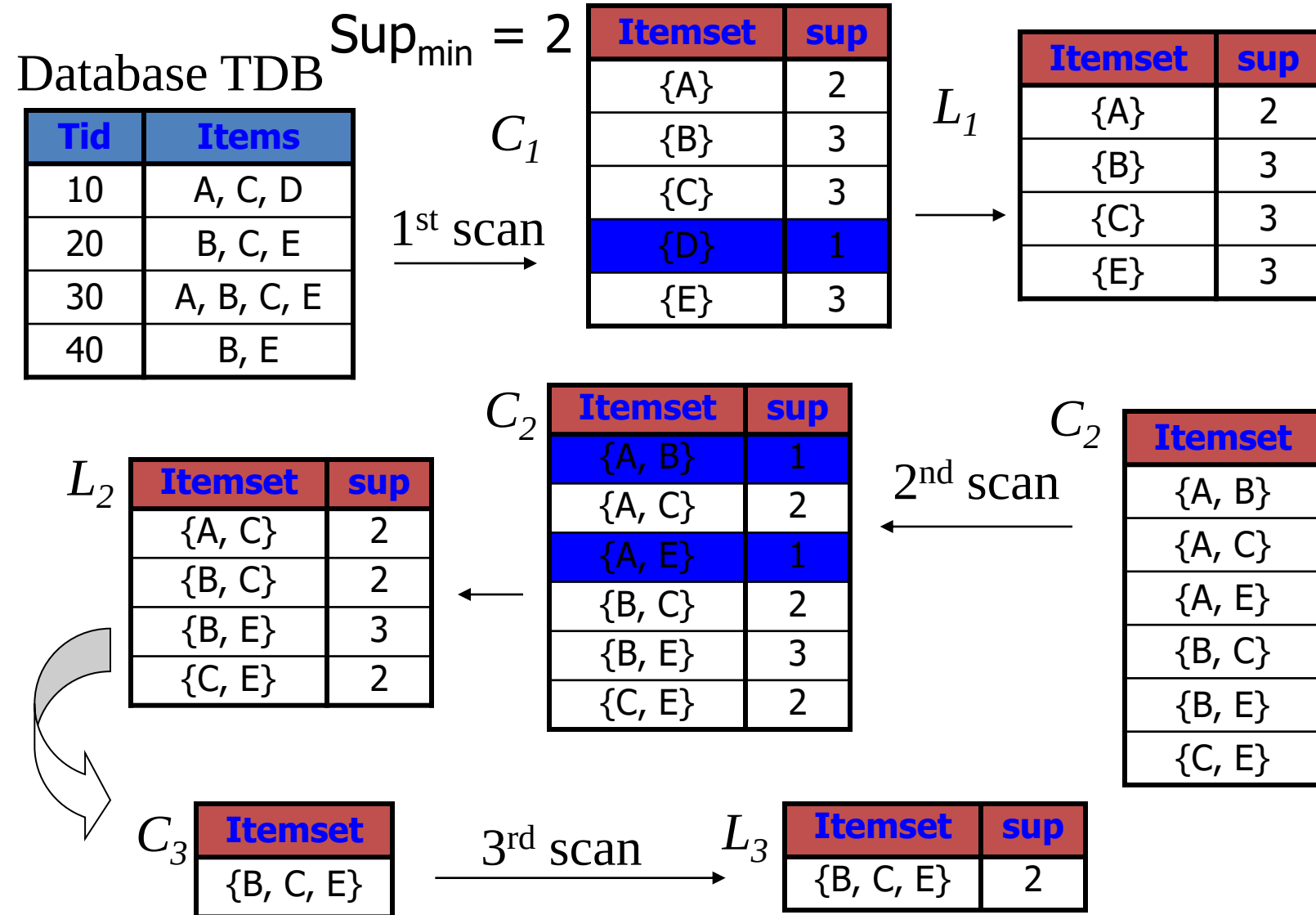
Method:

- Initially, scan DB once to get frequent 1-itemset
- Generate length $(k+1)$ candidate itemsets from length k frequent itemsets
- Test the candidates against DB
- Terminate when no frequent or candidate set can be generated

Steps in Apriori Algorithm



The Apriori Algorithm—An Example



The Apriori Algorithm (Pseudo-Code)

C_k : Candidate itemset of size k

L_k : frequent itemset of size k

$L_1 = \{\text{frequent items}\};$

for ($k = 1; L_k \neq \emptyset; k++$) **do begin**

C_{k+1} = candidates generated from L_k ;

for each transaction t in database do

increment the count of all candidates in C_{k+1} that are
contained in t

L_{k+1} = candidates in C_{k+1} with min_support

end

return $\cup_k L_k$;

Implementation of Apriori

- How to generate candidates?
 - Step 1: self-joining L_k
 - Step 2: pruning
- Example of Candidate-generation
 - $L_3 = \{abc, abd, acd, ace, bcd\}$
 - Self-joining: $L_3 * L_3$
 - $abcd$ from abc and abd
 - $acde$ from acd and ace
 - Pruning:
 - $acde$ is removed because ade is not in L_3
 - $C_4 = \{abcd\}$

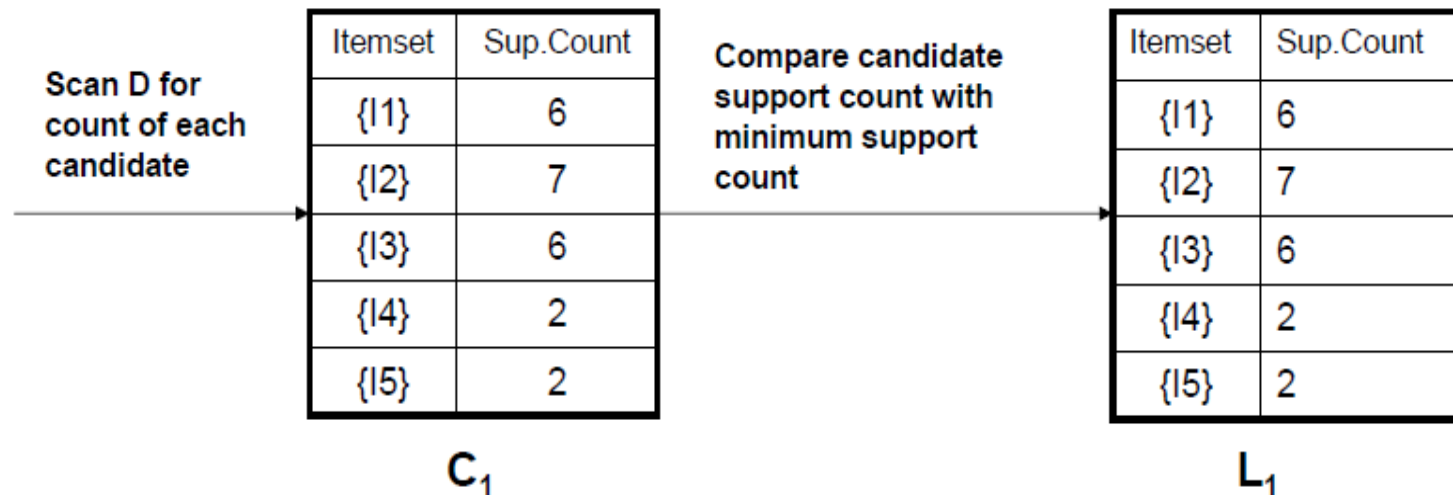
Apriori Example

- A Marketing manager of All-Electronics company branch would like to learn more about the buying habits of the customers to plan marketing strategies. Specifically he is interested to know “Which groups of items are customers likely to purchase on a given trip to the store?”. Consider the All-Electronics transaction database.

TID	List of item ID's
100	I1, I2,I5
101	I2,I4
102	I2,I3
103	I1,I2,I4
104	I1,I3
105	I2,I3
106	I1,I3
107	I1,I2,I3,I5
108	I1,I2,I3

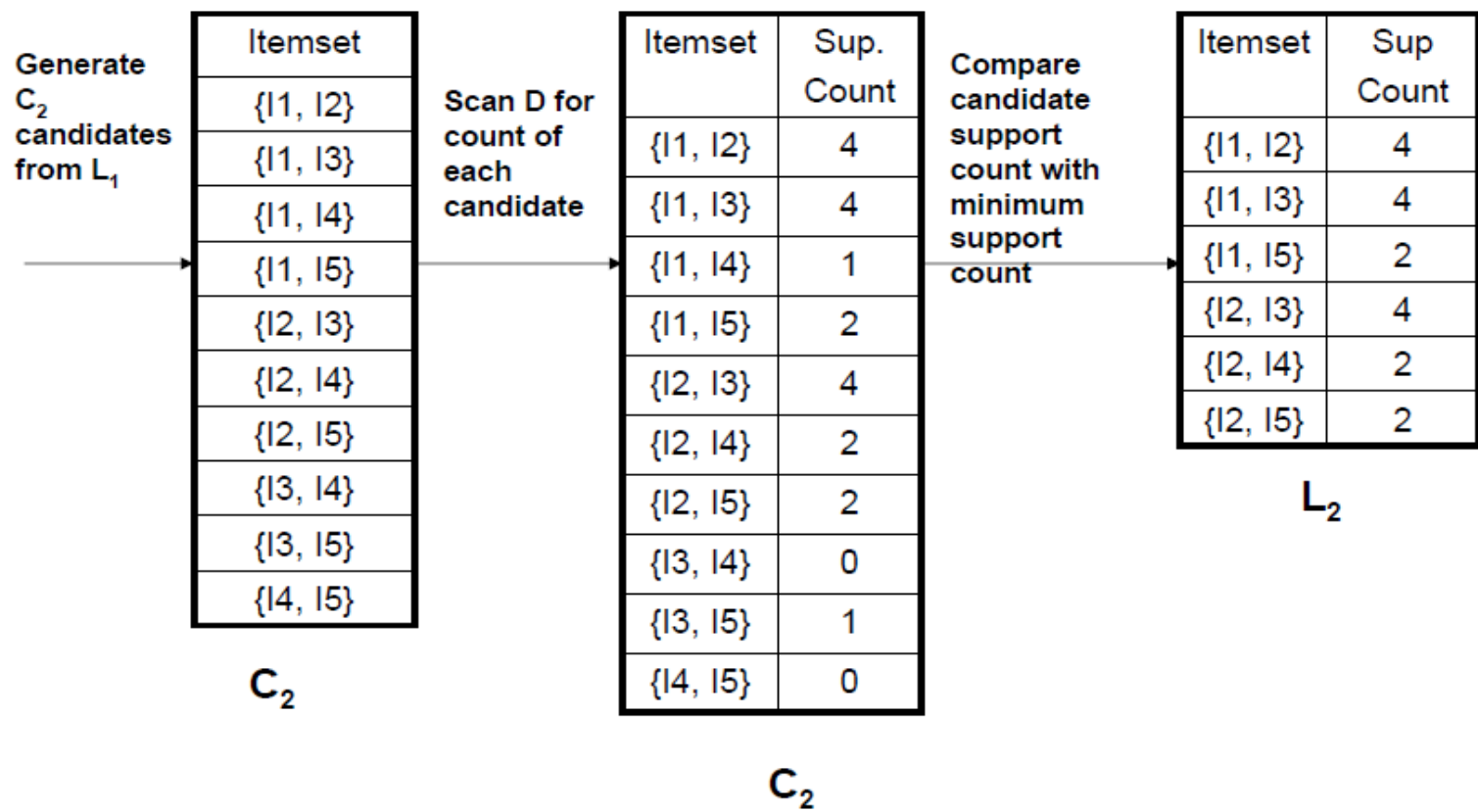
- Implement a algorithm to mine items frequently purchased together by customer with minimum support threshold for a item (or set of items) is 22% (2/9) i.e. (**min support = 2**) & minimum confidence required is **70%**.

Step 1: Generating 1-itemset Frequent Pattern



- The set of frequent 1-itemsets, L_1 , consists of the candidate 1-itemsets satisfying minimum support.
- In the first iteration of the algorithm, each item is a member of the set of candidate.

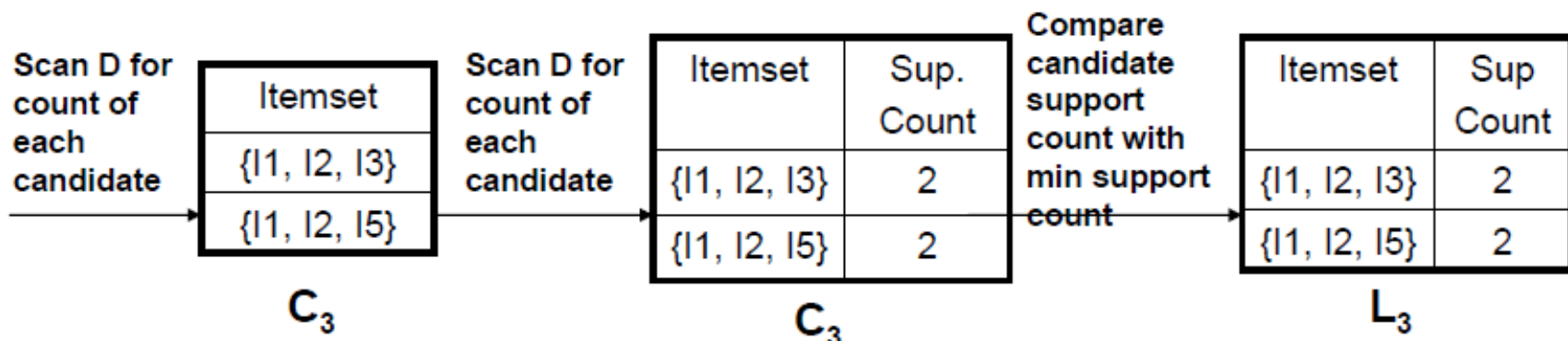
Step 2: Generating 2-itemset Frequent Pattern



Step 2: Generating 2-itemset Frequent Pattern

- To discover the set of frequent 2-itemsets, L_2 , the algorithm uses $L_1 \text{ Join } L_1$ to generate a candidate set of 2-itemsets, C_2 .
- Next, the transactions in D are scanned and the support count for each candidate itemset in C_2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L_2 , is then determined, consisting of those candidate 2-itemsets in C_2 having minimum support.
- Note: We haven't used Apriori Property yet.

Step 3: Generating 3-itemset Frequent Pattern



- The generation of the set of candidate 3-itemsets, C_3 , involves use of the Apriori Property.
- In order to find C_3 , we compute $L_2 \text{ Join } L_2$.
- $C_3 = L_2 \text{ Join } L_2 = \{\{l1, l2, l3\}, \{l1, l2, l5\}, \{l1, l3, l5\}, \{l2, l3, l4\}, \{l2, l3, l5\}, \{l2, l4, l5\}\}$.
- Now, Join step is complete and Prune step will be used to reduce the size of C_3 . Prune step helps to avoid heavy computation due to large C_k .

Step 3: Generating 3-itemset Frequent Pattern

- Based on the **Apriori property** that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?
- For example , lets take $\{I1, I2, I3\}$. The 2-item subsets of it are $\{I1, I2\}$, $\{I1, I3\}$ & $\{I2, I3\}$. Since all 2-item subsets of $\{I1, I2, I3\}$ are members of L_2 , We will keep $\{I1, I2, I3\}$ in C_3 .
- Lets take another example of $\{I2, I3, I5\}$ which shows how the pruning is performed. The 2-item subsets are $\{I2, I3\}$, $\{I2, I5\}$ & $\{I3, I5\}$.
- BUT, $\{I3, I5\}$ is not a member of L_2 and hence it is not frequent **violating Apriori Property**. Thus We will have to remove $\{I2, I3, I5\}$ from C_3 .
- Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after checking for all members of **result of Join operation for Pruning**.
- Now, the transactions in D are scanned in order to determine L_3 , **consisting of those candidates 3-itemsets in C_3 having minimum support**.

Step 4: Generating 4-itemset Frequent Pattern

- The algorithm uses $L_3 \text{ Join } L_3$ to generate a candidate set of 4-itemsets, C_4 . Although the join results in $\{\{I1, I2, I3, I5\}\}$, this itemset is pruned since its subset $\{\{I2, I3, I5\}\}$ is not frequent.
- Thus, $C_4 = \varnothing$, and algorithm terminates, having found all of the frequent items. This completes our Apriori Algorithm.
- What's Next ?
These frequent itemsets will be used to generate strong association rules (where strong association rules satisfy both minimum support & minimum confidence).

Step 5: Generating Association Rules from Frequent Itemsets

- Procedure:

- For each frequent itemset I , generate all nonempty subsets of I .
- For every nonempty subset s of I , output the rule " $s \rightarrow (I-s)$ " if $\text{support_count}(I) / \text{support_count}(s) \geq \text{min_conf}$ where min_conf is minimum confidence threshold.

- Back To Example:

We had $L = \{\{I1\}, \{I2\}, \{I3\}, \{I4\}, \{I5\}, \{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}, \{I1, I2, I3\}, \{I1, I2, I5\}\}$.

- Lets take $I = \{I1, I2, I5\}$.
- Its all nonempty subsets are $\{I1, I2\}, \{I1, I5\}, \{I2, I5\}, \{I1\}, \{I2\}, \{I5\}$.

Step 5: Generating Association Rules from Frequent Itemsets

- Let minimum confidence threshold is , say 70%.
- The resulting association rules are shown below, each listed with its confidence.
 - R1: $I1 \wedge I2 \rightarrow I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I2\} = 2/4 = 50\%$
 - R1 is Rejected.
 - R2: $I1 \wedge I5 \rightarrow I2$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I5\} = 2/2 = 100\%$
 - R2 is Selected.
 - R3: $I2 \wedge I5 \rightarrow I1$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I2, I5\} = 2/2 = 100\%$
 - R3 is Selected.

Step 5: Generating Association Rules from Frequent Itemsets

- R4: $I1 \rightarrow I2 \wedge I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1\} = 2/6 = 33\%$
 - R4 is Rejected.
- R5: $I2 \rightarrow I1 \wedge I5$
 - Confidence = $sc\{I1, I2, I5\} / \{I2\} = 2/7 = 29\%$
 - R5 is Rejected.
- R6: $I5 \rightarrow I1 \wedge I2$
 - Confidence = $sc\{I1, I2, I5\} / \{I5\} = 2/2 = 100\%$
 - R6 is Selected.

In this way, We have found three strong association rules.

Improving the Efficiency of Apriori

▶ Major computational challenges

- Huge number of candidates
- Multiple scans of transaction database
- Tedious workload of support counting for candidates

▶ Improving Apriori: general ideas

- Shrink number of candidates
- Reduce passes of transaction database scans
- Facilitate support counting of candidates

FP-growth: Frequent Pattern-Growth

- ▶ Adopts a divide and conquer strategy
- ▶ Compress the database representing frequent items into a **frequent -pattern tree** or **FP-tree**
 - Retains the itemset association information
- ▶ Divide the compressed database into a set of conditional databases, each associated with one frequent item
- ▶ Mine each such databases separately

Example: FP-growth

- ▶ The first scan of data is the same as Apriori
- ▶ Derive the set of frequent 1-itemsets
- ▶ Let $\text{min-sup}=2$
- ▶ Generate a set of ordered items

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2

Transactional Database

TID	List of item IDS
T100	I1,I2,I5
T200	I2,I4
T300	I2,I3
T400	I1,I2,I4
T500	I1,I3
T600	I2,I3
T700	I1,I3
T800	I1,I2,I3,I5
T900	I1,I2,I3

Construct the FP-Tree

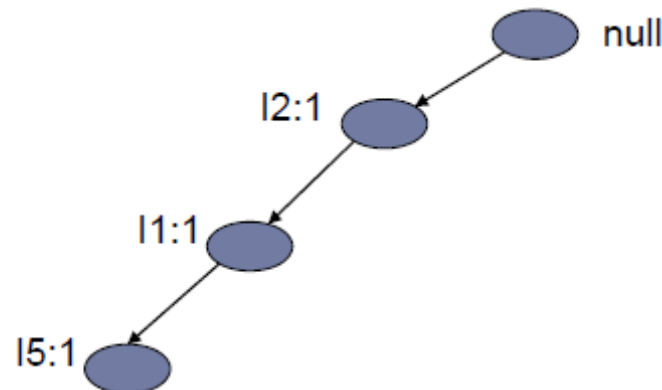
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

- 1- Order the items T100: {I2,I1,I5}
- 2- Construct the first branch: <I2:1>, <I1:1>, <I5:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

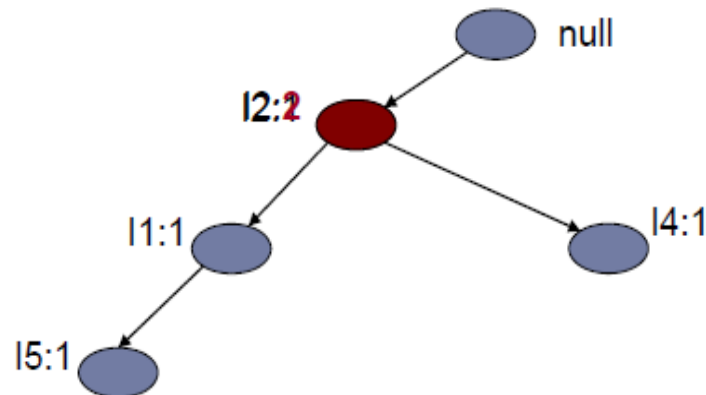
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

- 1- Order the items T200: {I2,I4}
- 2- Construct the second branch: <I2:1>, <I4:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

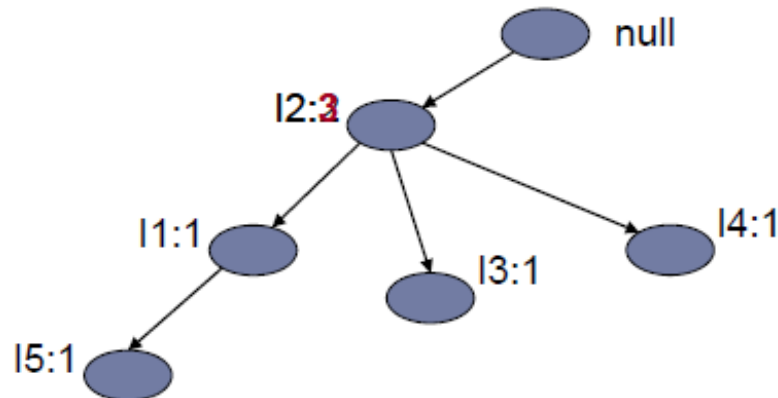
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

- 1- Order the items T300: {I2,I3}
- 2- Construct the third branch: <I2:2>, <I3:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

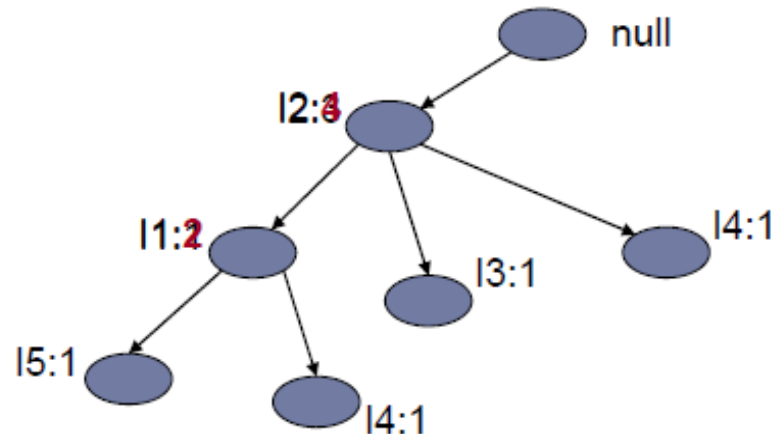
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

- 1- Order the items T400: {I2,I1,I4}
- 2- Construct the fourth branch: <I2:3>, <I1:1>, <I4:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

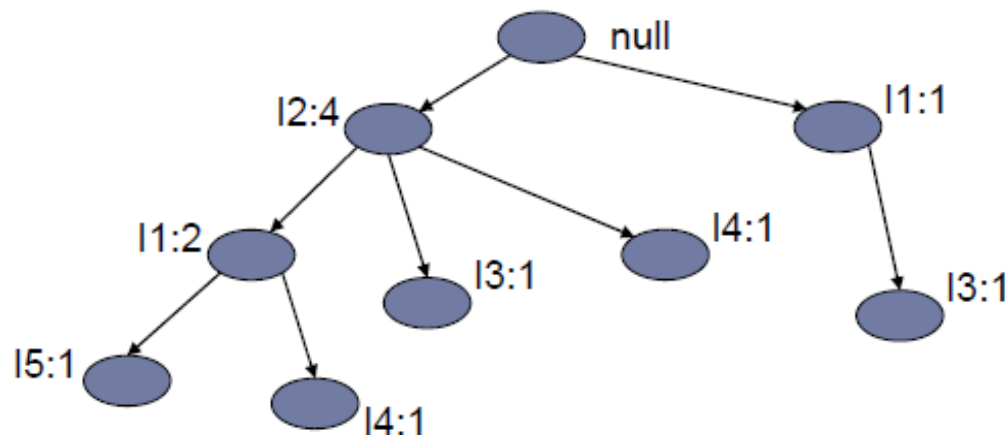
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

- 1- Order the items T400: {I1,I3}
- 2- Construct the fifth branch: $\langle I1:1 \rangle, \langle I3:1 \rangle$

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2

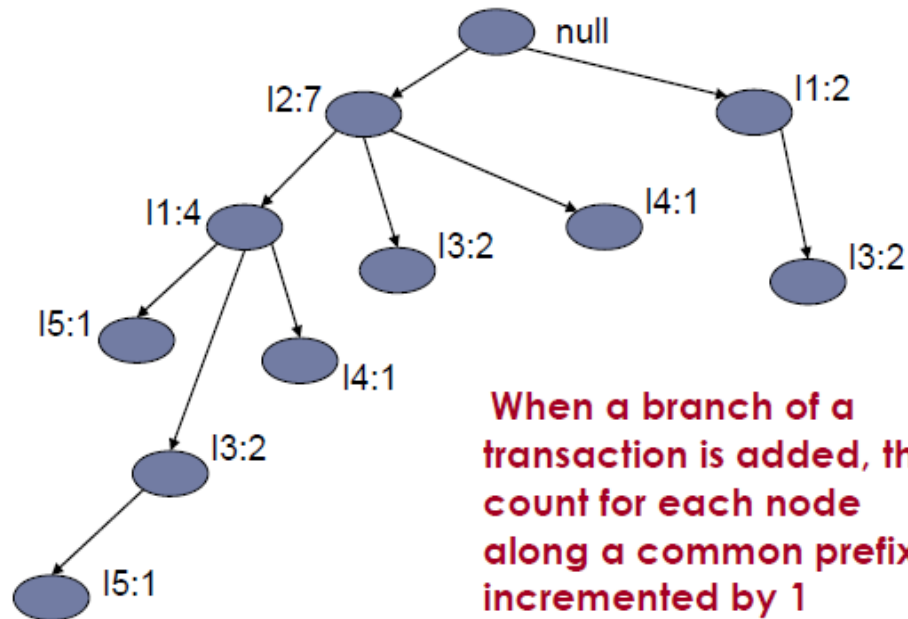


Construct the FP-Tree

Transactional Database

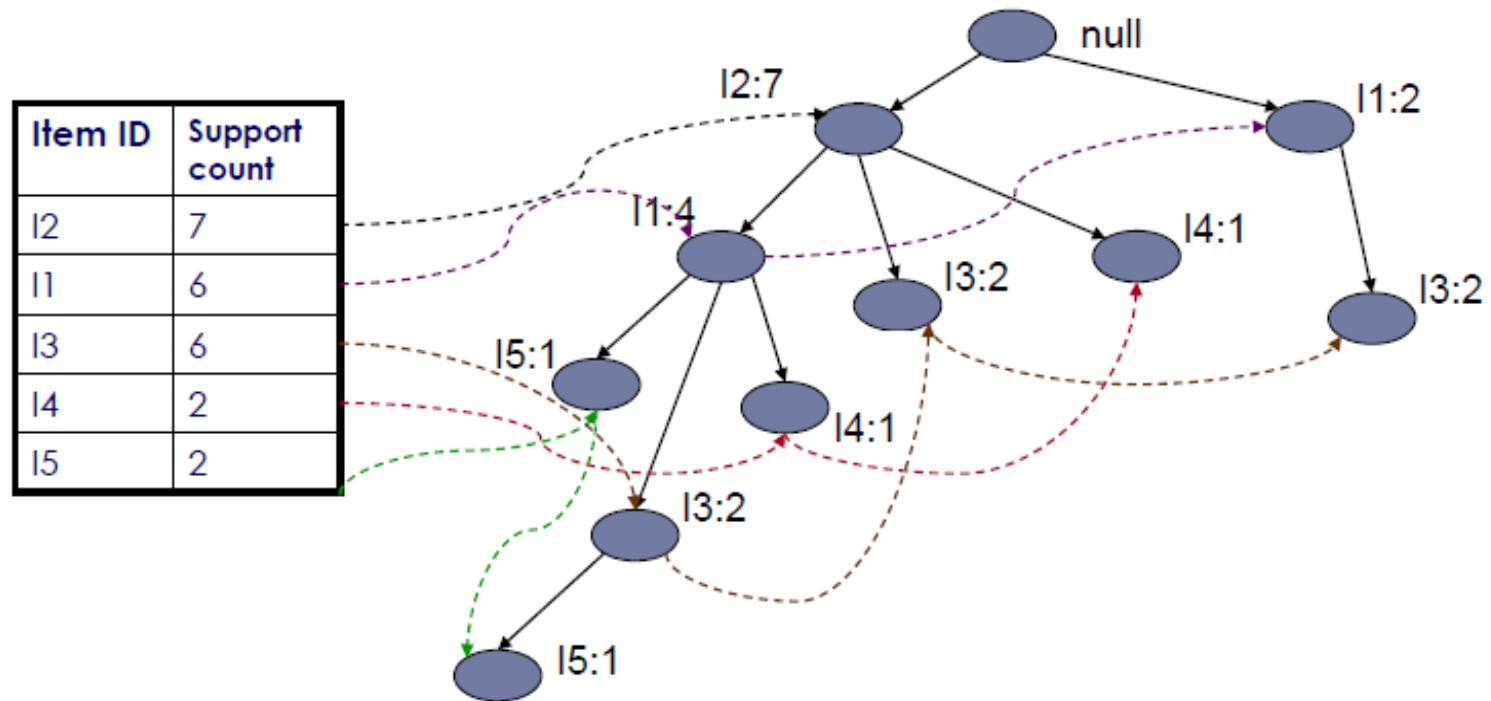
TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



When a branch of a transaction is added, the count for each node along a common prefix is incremented by 1

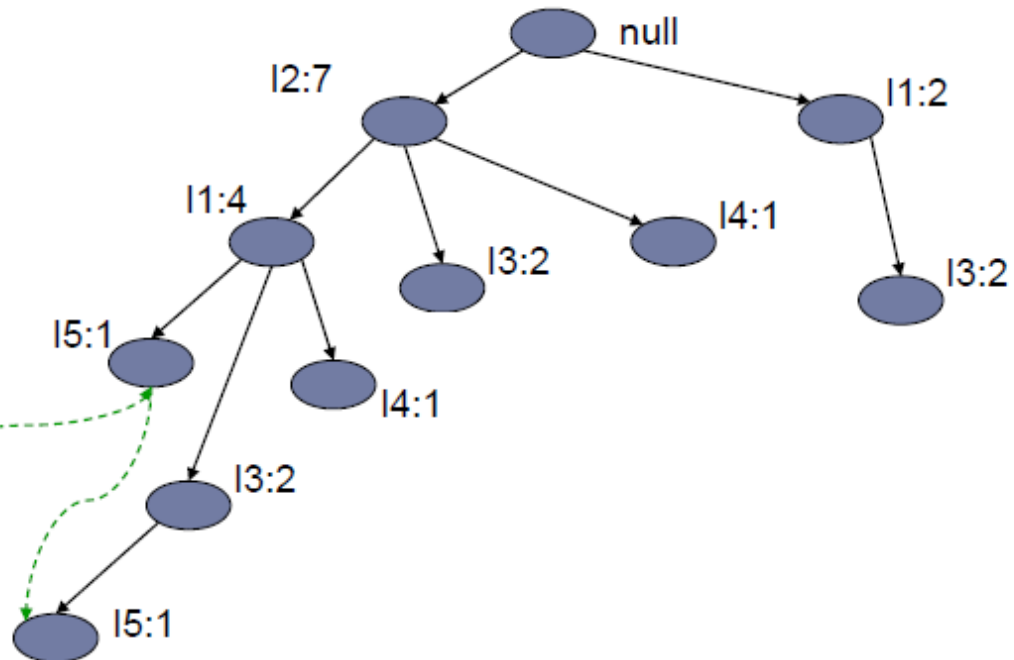
Construct the FP-Tree



The problem of mining frequent patterns in databases is transformed to that of mining the FP-tree

Construct the FP-Tree

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



-Occurrences of I5: <I2,I1,I5> and <I2,I1,I3,I5>

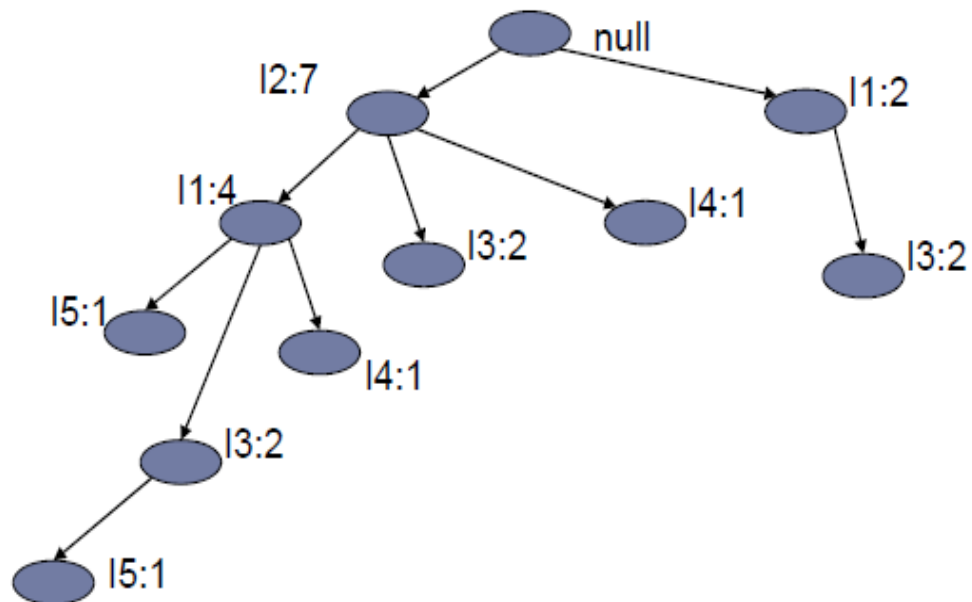
-Two prefix Paths <I2, I1: 1> and <I2,I1,I3: 1>

-Conditional FP tree contains only <I2: 2, I1: 2>, I3 is not considered because its support count of 1 is less than the minimum support count.

-Frequent patterns {I2,I5:2}, {I1,I5:2},{I2,I1,I5:2}

Construct the FP-Tree

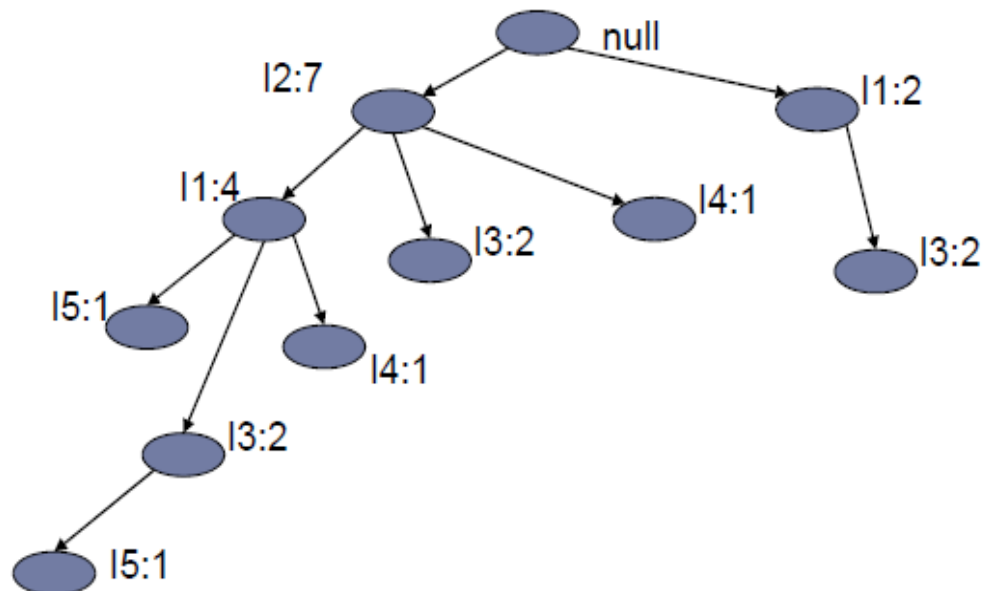
Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



TID	Conditional Pattern Base	Conditional FP-tree
I5	$\{\{I2, I1:1\}, \{I2, I1, I3:1\}\}$	$\langle I2:2, I1:2 \rangle$
I4	$\{\{I2, I1:1\}, \{I2, I1\}\}$	$\langle I2:2 \rangle$
I3	$\{\{I2, I1:2\}, \{I2:2\}, \{I1:2\}\}$	$\langle I2:4, I1:2 \rangle, \langle I1:2 \rangle$
I1	$\{I2:4\}$	$\langle I2:4 \rangle$

Construct the FP-Tree

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



TID	Conditional FP-tree	Frequent Patterns Generated
I5	<I2:2,I1:2>	{I2,I5:2}, {I1,I5:2},{I2,I1,I5:2}
I4	<I2:2>	{I2,I4:2}
I3	<I2:4,I1:2>,<I1:2>	{I2,I3:4},{I1,I3:4},{I2,I1,I3:2}
I1	<I2:4>	{I2,I1:4}

FP-growth properties

- ▶ FP-growth transforms the problem of finding long frequent patterns to searching for shorter ones recursively and concatenating the suffix
- ▶ It uses the least frequent suffix offering a good selectivity
- ▶ It reduces the search cost
- ▶ If the tree does not fit into main memory, partition the database
- ▶ Efficient and scalable for mining both long and short frequent patterns

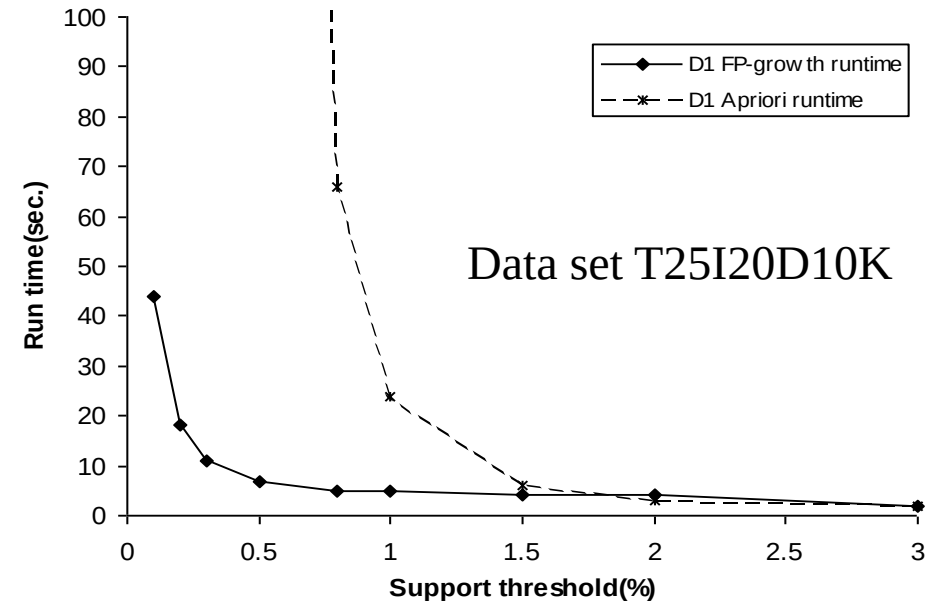
Benefits of the FP-tree Structure

- Completeness
 - Preserve complete information for frequent pattern mining
 - Never break a long pattern of any transaction
- Compactness
 - Reduce irrelevant info—infrequent items are gone
 - Items in frequency descending order: the more frequently occurring, the more likely to be shared
 - Never be larger than the original database (not count node-links and the *count* field)

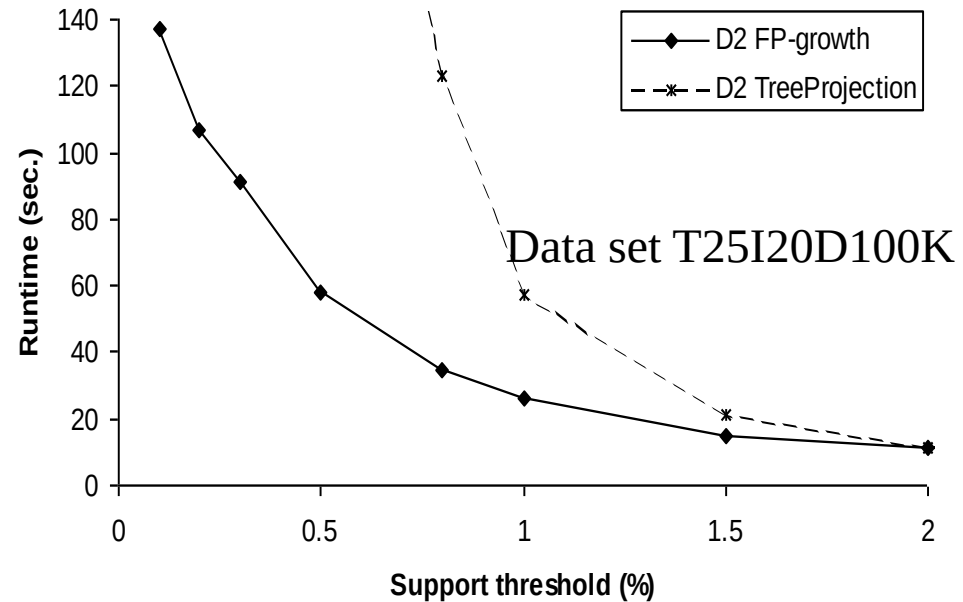
Scaling FP-growth by Database Projection

- What about if FP-tree cannot fit in memory?
 - DB projection
- First partition a database into a set of projected DBs
- Then construct and mine FP-tree for each projected DB
- **Parallel projection** vs. **partition projection** techniques
 - Parallel projection
 - Project the DB in parallel for each frequent item
 - Parallel projection is space costly
 - All the partitions can be processed in parallel
 - Partition projection
 - Partition the DB based on the ordered frequent items
 - Passing the unprocessed parts to the subsequent partitions

Performance of FP Growth in Large Datasets



FP-Growth vs. Apriori



FP-Growth vs. Tree-Projection

Advantages of the Pattern Growth Approach

- Divide-and-conquer:
 - Decompose both the mining task and DB according to the frequent patterns obtained so far
 - Lead to focused search of smaller databases
- Other factors
 - No candidate generation, no candidate test
 - Compressed database: FP-tree structure
 - No repeated scan of entire database
 - Basic ops: counting local freq items and building sub FP-tree, no pattern search and matching
- A good open-source implementation and refinement of FPGrowth
 - FPGrowth+ (Grahne and J. Zhu, FIMI'03)

Hash Based Algorithm

Similar to Apriori. Uses hash to reduce the size of set C 's. This improves the computing efficiency while the number of candidate item sets to be checked is reduced.

Hash Based Algorithm DHP

- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$

Hash Based Algorithm DHP

- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$

T	
$T_1 = (A, B, C)$	$\{A, B\} \{A, C\} \{B, C\}.$
$T_2 = (A, F)$	$\{A, F\}$
$T_3 = (A, B, C, E)$	$\{A, B\} \{A, C\} \{A, E\} \{B, C\} \{B, E\} \{C, E\}.$
$T_4 = (A, B, D, F)$	-
$T_5 = (C, F)$	-
$T_6 = (A, B, C)$	-
$T_7 = (A, B, C, E)$	-
$T_8 = (C, D, E)$	-
$T_9 = (B, D, E)$	-

Hash Based Algorithm DHP

- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$

T	
$T_1 = (A, B, C)$	$\{A, B\} \{A, C\} \{B, C\}$
$T_2 = (A, F)$	$\{A, F\}$
$T_3 = (A, B, C, E)$	$\{A, B\} \{A, C\} \{A, E\} \{B, C\} \{B, E\} \{C, E\}$
$T_4 = (A, B, D, F)$	-
$T_5 = (C, F)$	-
$T_6 = (A, B, C)$	-
$T_7 = (A, B, C, E)$	-
$T_8 = (C, D, E)$	-
$T_9 = (B, D, E)$	-

hash	Bucket
0	$\{C, E\} \{A, D\} \{C, E\}$ (4)
1	$\{A, E\} \{C, F\} \{A, E\}$ (3)
2	$\{B, C\} \{A, F\} \{B, C\} \{A, F\} \{B, C\} \{B, C\} \{D, E\} \{D, E\}$ (8)
3	$\{B, D\} \{B, D\}$ (2)
4	$\{B, E\} \{D, F\} \{B, E\} \{B, E\}$ (4)
5	$\{A, B\} \{A, B\} \{B, F\} \{B, A\} \{A, B\}$ (5)
6	$\{A, C\} \{A, C\} \{A, C\} \{A, C\} \{C, D\}$ (5)

- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$

T	
$T_1 = (A, B, C)$	$\{A, B\} \{A, C\} \{B, C\}$
$T_2 = (A, F)$	$\{A, F\}$
$T_3 = (A, B, C, E)$	$\{A, B\} \{A, C\} \{A, E\} \{B, C\} \{B, E\} \{C, E\}$
$T_4 = (A, B, D, F)$	-
$T_5 = (C, F)$	-
$T_6 = (A, B, C)$	-
$T_7 = (A, B, C, E)$	-
$T_8 = (C, D, E)$	-
$T_9 = (B, D, E)$	-

hash	Bucket
0	$\{C, E\} \{A, D\} \{C, E\}$ (4)
1	$\{A, E\} \{C, F\} \{A, E\}$ (3)
2	$\{B, C\} \{A, F\} \{B, C\} \{A, F\} \{B, C\} \{B, C\} \{D, E\} \{D, E\}$ (8)
3	$\{B, D\} \{B, D\}$ (2)
4	$\{B, E\} \{D, F\} \{B, E\} \{B, E\}$ (4)
5	$\{A, B\} \{A, B\} \{B, F\} \{B, A\} \{A, B\}$ (5)
6	$\{A, C\} \{A, C\} \{A, C\} \{A, C\} \{C, D\}$ (5)

Note that the total counts for buckets 1 and 3 cannot satisfy the minimum support constraint, therefore, $\{A, E\}$, should not be included in C_2 .

Create hash table H_2 using hash function $h(x, y) = ((\text{order of } x) \times 10 + (\text{order of } y)) \bmod 7$

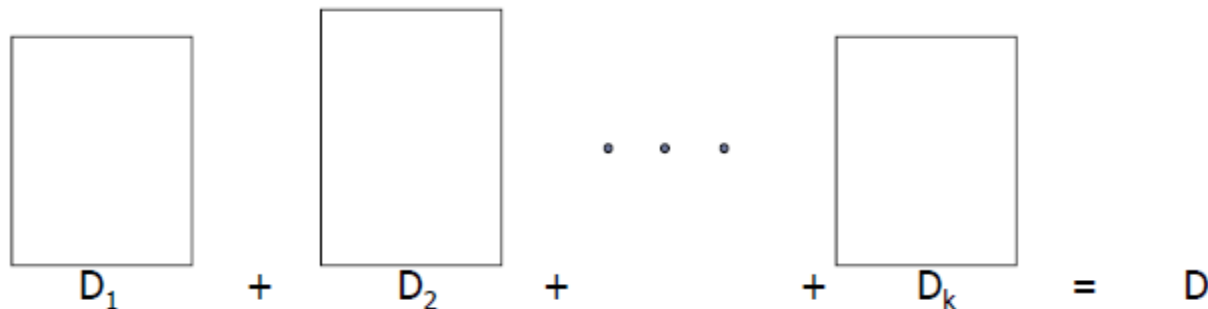
H_2							
bucket address	0	1	2	3	4	5	6
bucket count	2	2	4	2	2	4	4
bucket contents	{I1, I4} {I3, I5}	{I1, I5} {I1, I5}	{I2, I3} {I2, I3} {I2, I3} {I2, I3}	{I2, I4} {I2, I4}	{I2, I5} {I2, I5}	{I1, I2} {I1, I2} {I1, I2}	{I1, I3} {I1, I3} {I1, I3}

If the minimum support count is, say, 3, then the itemsets in buckets 0, 1, 3, and 4 cannot be frequent and so they should not be included in C_2 .

Hash-based technique (hashing itemsets into corresponding buckets): A hash-based technique can be used to reduce the size of the candidate k -itemsets, C_k , for $k > 1$. For example, when scanning each transaction in the database to generate the frequent 1-itemsets, L_1 , we can generate all the 2-itemsets for each transaction, hash (i.e., map) them into the different *buckets* of a *hash table* structure, and increase the corresponding bucket counts. A 2-itemset with a corresponding bucket count in the hash table that is below the support threshold cannot be frequent and thus should be removed from the candidate set. Such a hash-based technique may substantially reduce the number of candidate k -itemsets examined (especially when $k = 2$).

Partition: Scan Database Only Twice

- ▶ Subdivide the transactions of **D** into **k** non overlapping partitions
- ▶ Any itemset that is potentially frequent in **D** must be frequent in at least one of the partitions **D_i**
- ▶ Each partition can fit into main memory, thus it is read only once
- ▶ **Steps:**
 - Scan1: partition database and find local frequent patterns
 - Scan2: consolidate global frequent patterns



Partition Based Algorithm

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$ must be a frequent item set in at least one of the n partitions

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$ must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \dots, p_n$ to find local frequent item sets

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \dots, p_n$ to find local frequent item sets Set of candidate item sets is constructed by taking the union

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \dots, p_n$ to find local frequent item
- sets Set of candidate item sets is constructed by taking the union Scans the D second time to calculate the support of each item set

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \dots, p_n$ to find local frequent item sets
- Set of candidate item sets is constructed by taking the union
- Scans the D second time to calculate the support of each item

A handwritten table with 9 rows and 2 columns. The first column lists partitions P1, P2, and P3. The second column lists item sets T1 through T9. Red horizontal lines separate the rows for each partition.

P1	$T_1 = (A, B, C)$
	$T_2 = (A, F)$
	$T_3 = (A, B, C, E)$
	$T_4 = (A, B, D, F)$
P2	$T_5 = (C, F)$
	$T_6 = (A, B, C)$
	$T_7 = (A, B, C, E)$
P3	$T_8 = (C, D, E)$
	$T_9 = (B, D, E)$

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \dots, p_n$ to find local frequent item
- sets Set of candidate item sets is constructed by taking the union Scans the D second time to calculate the support of each item

P1	$T_1 = (A, B, C)$
	$T_2 = (A, F)$
	$T_3 = (A, B, C, E)$
P2	$T_4 = (A, B, D, F)$
	$T_5 = (C, F)$
	$T_6 = (A, B, C)$
P3	$T_7 = (A, B, C, E)$
	$T_8 = (C, D, E)$
	$T_9 = (B, D, E)$

Partition	Frequent Item set
P_1	$\{A\} \{B\} \{C\} \{AB\} \{AC\} \{BC\} \{ABC\}$
P_2	$\{A\} \{B\} \{C\} \{F\} \{AB\}$
P_3	$\{B\} \{C\} \{D\} \{E\} \{BE\} \{CE\} \{DE\}$

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \dots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \dots, p_n$ to find local frequent item sets
- Set of candidate item sets is constructed by taking the union
- Scans the D second time to calculate the support of each item

P1	$T_1 = (A, B, C)$
	$T_2 = (A, F)$
	$T_3 = (A, B, C, E)$
P2	$T_4 = (A, B, D, F)$
	$T_5 = (C, F)$
	$T_6 = (A, B, C)$
P3	$T_7 = (A, B, C, E)$
	$T_8 = (C, D, E)$
	$T_9 = (B, D, E)$

Partition	Frequent item set
P_1	$\{A\} \{B\} \{C\} \{AB\} \{AC\} \{BC\} \{ABC\}$
P_2	$\{A\} \{B\} \{C\} \{F\} \{AB\}$
P_3	$\{B\} \{C\} \{D\} \{E\} \{BE\} \{CE\} \{DE\}$

Union is taken

Candidate item sets
$\{A\} \{B\} \{C\} \{D\} \{E\} \{F\} \{AB\} \{AC\}$
$\{BC\} \{CE\} \{DE\} \{ABC\} \{ABC\}$

Second scan of D examines the required support

Incremental Association Rule Mining

- Real databases are generally dynamic (not static)

Incremental Association Rule Mining

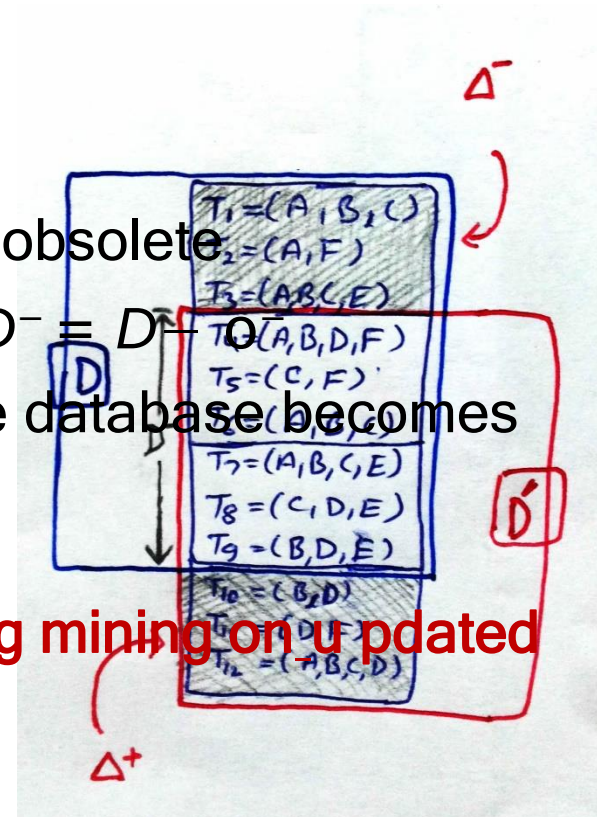
- Real databases are generally dynamic (not static)
- New transactions are generated and old transactions may be obsolete over time

- Real databases are generally dynamic (not static)
- New transactions are generated and old transactions may be obsolete over time

Primitives:

- Let D be the initial database
- o^- portion of the database becomes obsolete
- Therefore, the database is left with $D^- = D - o^-$
- o^+ more transactions are added. The database becomes $D' = D^- + o^+ = D - o^- + o^+$

Incremental update can avoid redoing mining on updated database



- 1 The update problem can be reduced to finding the new set of frequent item sets. After that, the new association rules can be computed from the new frequent item sets.
- 2 An old frequent item set has the potential to become infrequent in the updated database.
- 3 Similarly, an old infrequent item set could become frequent in the new database.
- 4 In order to find the new frequent item sets “exactly”, all the records in the updated database, including those from the original database, have to be checked against every candidate set.

We would evaluate two algorithms *viz.* FUP and FUP2

Algorithm FUP

- FUP stands for **F**ast **U**Pdate. Can handle insertions only
- Specifically, given the original database D and its corresponding frequent item set $L = \{L_1, L_2, \dots, L_k\}$.
- The goal is to reuse the information to efficiently obtain $L' = \{L'_1, L'_2, \dots, L'_k\}$ for new database $D' = D \cup o^+$

By utilizing the definition of support and constraint of minimum support S_{min} , following is used by FUP.

An original frequent item set $X \in L$, becomes infrequent in D^J iff

- $support(X)_{D'} < S_{min}$

An item set $X \notin L$, becomes frequent in D^J iff $support(X)_{o^+} \geq$

- S_{min}
- If a k -item set X whose $(k - 1)$ -subset(s) becomes infrequent, i.e., the subset is in L_{k-1} but not in L_{k-1}^J , then X must be infrequent in D^J .

FUP at work

Consider the database D and the related frequent set discovered with Apriori

$T_1 = (A, B, C)$
 $T_2 = (A, F)$
 $T_3 = (A, B, C, E)$
 $T_4 = (A, B, D, F)$
 $T_5 = (C, F)$
 $T_6 = (A, B, C)$
 $T_7 = (A, B, C, E)$
 $T_8 = (C, D, E)$
 $T_9 = (B, D, E)$

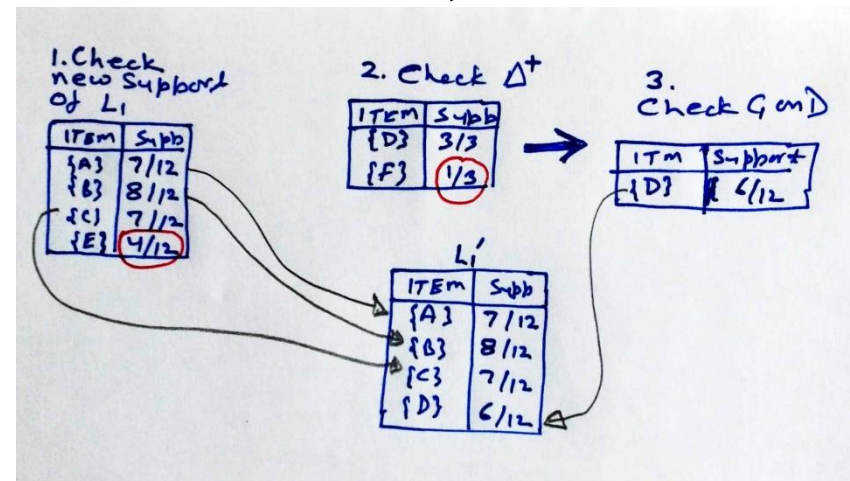
Item set	Support
{A}	6/9
{B}	6/9
{C}	6/9
{E}	4/9
{AB}	5/9
{AC}	4/9
{BC}	4/9
{ABC}	4/9

Consider the arrival of o^+ more transactions

$T_1 = (A, B, C)$
 $T_2 = (A, F)$
 $T_3 = (A, B, C, E)$
 $T_4 = (A, B, D, F)$
 $T_5 = (C, F)$
 $T_6 = (A, B, C)$
 $T_7 = (A, B, C, E)$
 $T_8 = (C, D, E)$
 $T_9 = (B, D, E)$

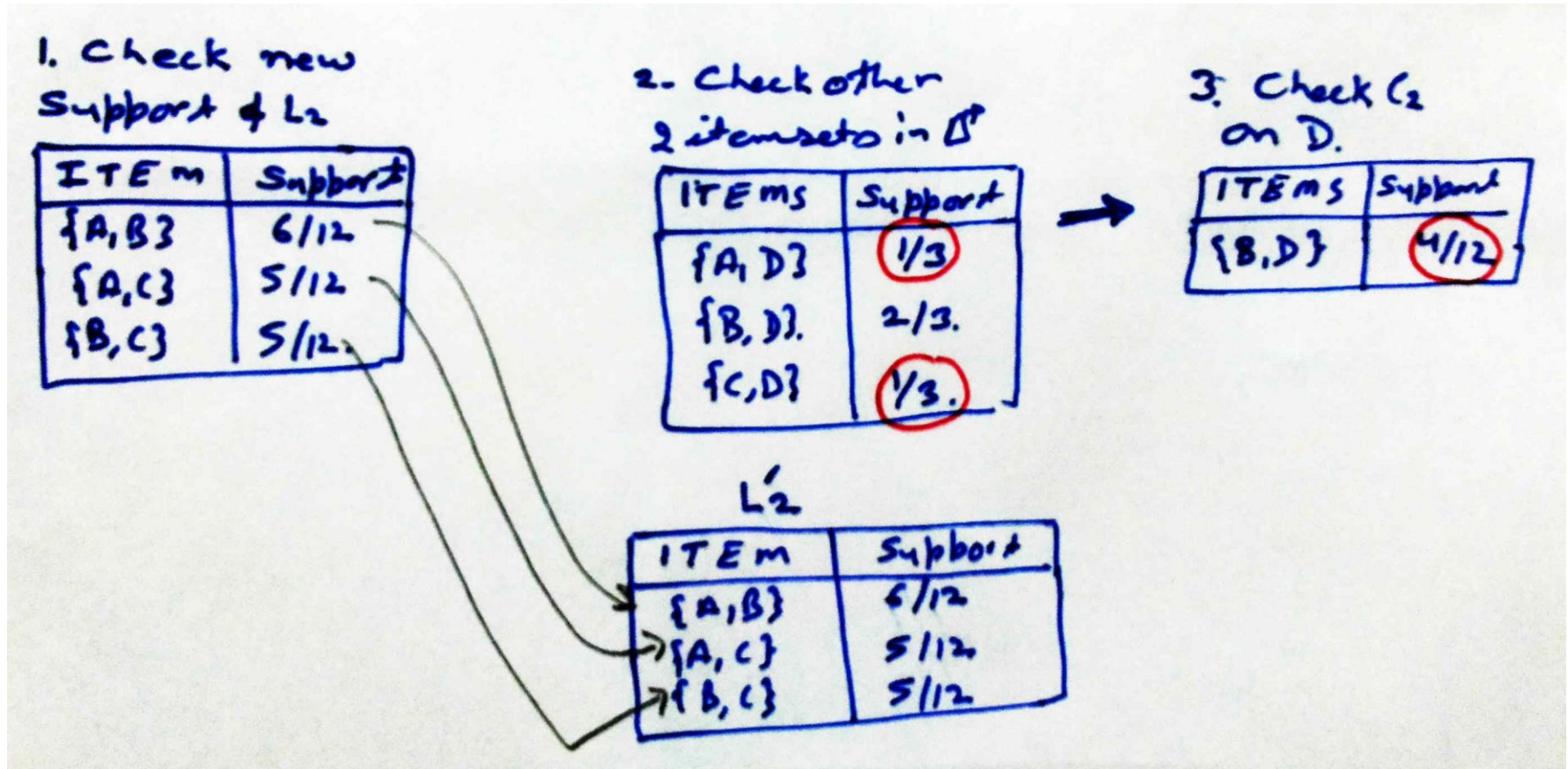
$\Delta^+ \rightarrow$
 $T_{10} = (B, D)$
 $T_{11} = (D, F)$
 $T_{12} = (A, B, C, D)$

The first iteration, is as below.



FUP at work (contd...)

The second iteration, is as below.

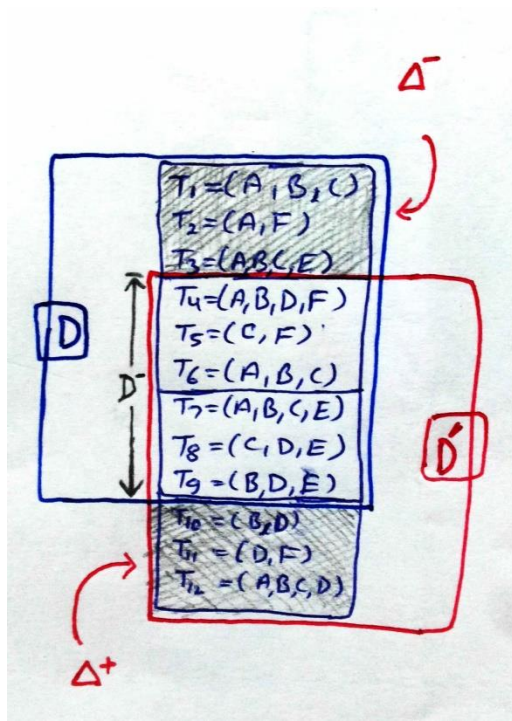


Similarly it is executed for next levels.

FUP₂

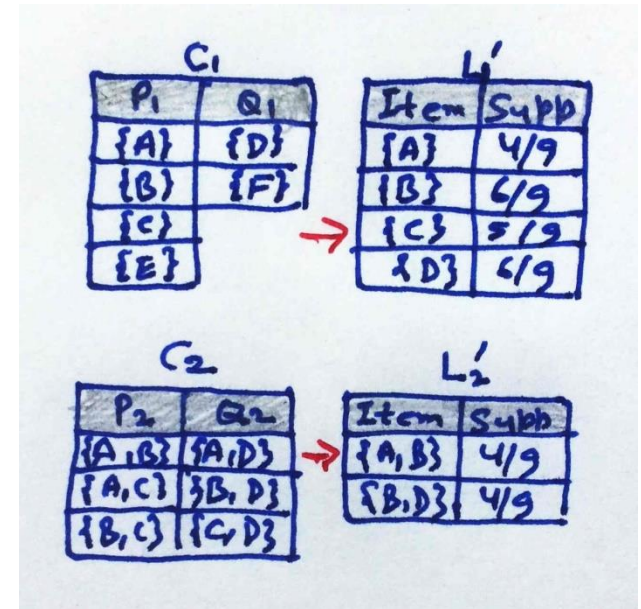
- FUP₂ can work for both o^- and o^+
- L_k from previous mining result is used for dividing candidate itemset C_k into two parts
 - › $P_k = C_k \cap L_k$
 - › $Q_k = C_k - P_k$
- Itemset that is frequent in o^- , must be infrequent in D^- .
- Further if itemset in Q_k is infrequent in o^+ then it is infrequent in D^- .
- This technique helps to effectively reduce number of candidate itemsets.

FUP₂ at work



Item set	Support
{A}	6/9
{B}	6/9
{C}	6/9
{E}	4/9
{AB}	5/9
{AC}	4/9
{BC}	4/9
{ABC}	4/9

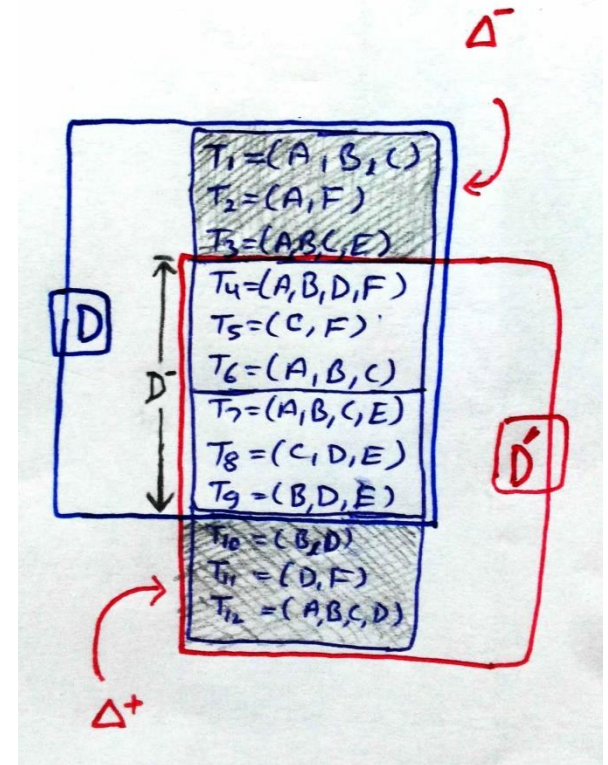
Frequent itemsets of D



- C_1 is set of all items. It is divided in P_i and Q_i
- Being frequent, support for all items in P_i is known. It could be updated using o^- and o^+ only.
- $\text{Count}(\{A\})_{D'} = \text{Count}(\{A\})_D - \text{Count}(\{A\})_{o^-} + \text{Count}(\{A\})_{o^+}$
 $= 6 - 3 + 1 = 4$

FUP₂ at work

- In some cases only the scan of o^- and o^+ is required.
- For example, $\text{Count}(\{F\})_{o^+} - \text{Count}(\{F\})_{o^-} = 0$ showing that support of $\{F\}$ can not be improved.
- Consequently, fewer itemsets have to be further scanned
- An iteration finishes when all the itemsets in P_i and Q_i are verified, and new set of frequent itemsets L^J_i is generated



FUP₂H

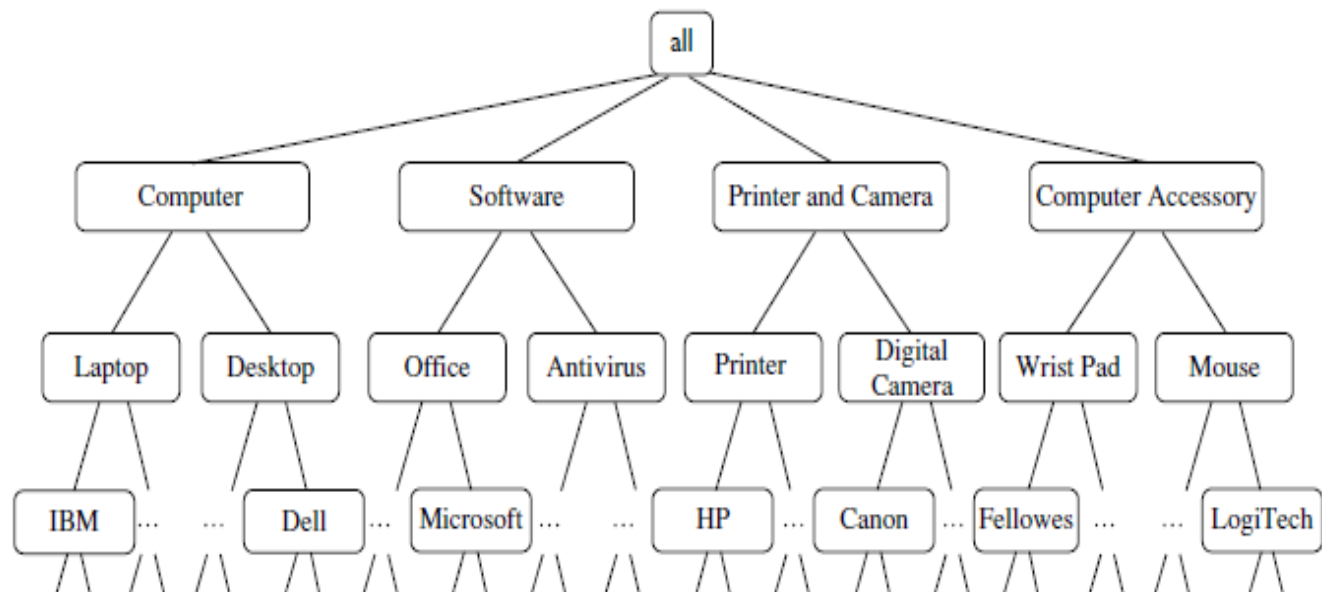
Uses hashing for performance improvement

Mining Multilevel Associations

- For many applications, strong associations discovered at high abstraction levels, though with high support, could be common sense knowledge.
- We may want to drill down to find novel patterns at more detailed levels. On the other hand, there could be too many scattered patterns at low or primitive abstraction levels, some of which are just trivial specializations of patterns at higher levels.
- Therefore, it is interesting to examine how to develop effective methods for mining patterns at multiple abstraction levels, with sufficient flexibility for easy traversal among different abstraction spaces.

Table 7.1 Task-Relevant Data, *D*

<i>TID</i>	<i>Items Purchased</i>
T100	Apple 17" MacBook Pro Notebook, HP Photosmart Pro b9180
T200	Microsoft Office Professional 2010, Microsoft Wireless Optical Mouse 5000
T300	Logitech VX Nano Cordless Laser Mouse, Fellowes GEL Wrist Rest
T400	Dell Studio XPS 16 Notebook, Canon PowerShot SD1400
T500	Lenovo ThinkPad X200 Tablet PC, Symantec Norton Antivirus 2010
...	...

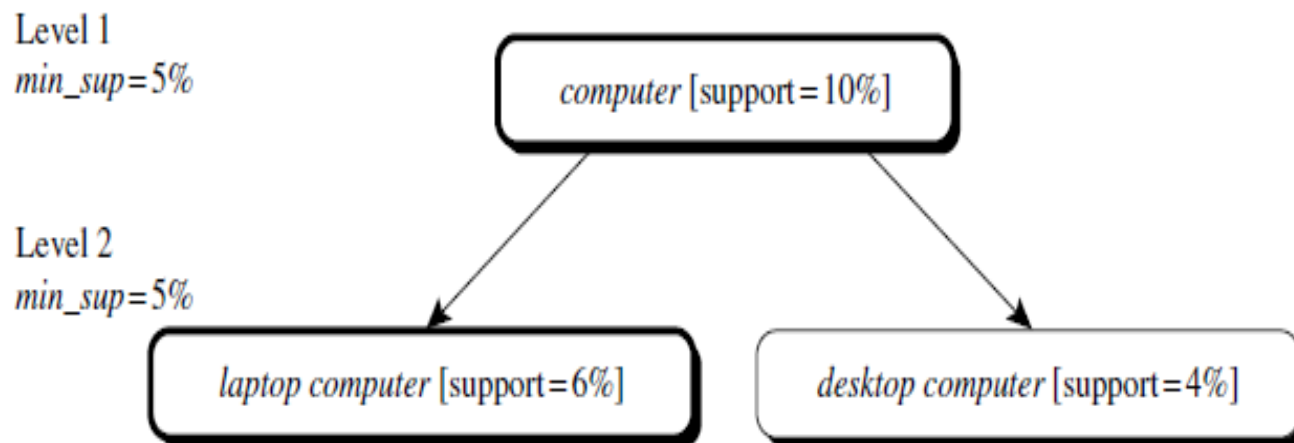


Concept hierarchy for *AllElectronics* computer items.

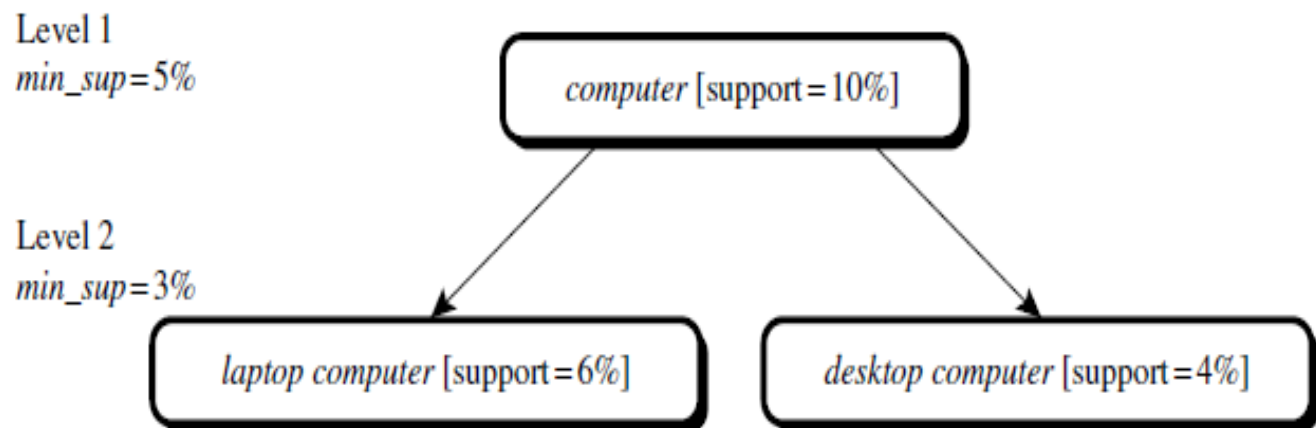
- Suppose we are given the task-relevant set of transactional data in Table for sales in an *AllElectronics* store, showing the items purchased
- For each transaction. The concept hierarchy for the items is shown in Figure, A concept hierarchy defines a sequence of mappings from a set of low-level concepts to a higher-level, more general concept set.
- Data can be generalized by replacing low-level concepts within the data by their corresponding higher-level concepts, or *ancestors*, from a concept hierarchy.
- Concept hierarchy has five levels, respectively referred to as levels 0 through 4, starting with level 0 at the root node for all (the most general abstraction level).
- Here, level 1 includes *computer, software, printer and camera*, and *computer accessory*; level 2 includes *laptop computer, desktop computer, office software, antivirus software*, etc.; and level 3 includes *Dell desktop computer, . . . , Microsoft office software*, etc.
- Level 4 is the most specific abstraction level of this hierarchy.

- For this example, the concept hierarchy is generated from data on product specifications. Concept hierarchies for numeric attributes can be generated using discretization techniques.
- The items in Table are at the lowest level of Figure concept hierarchy. It is difficult to find interesting purchase patterns in such raw or primitive-level data.
- For instance, if “*Dell Studio XPS 16 Notebook*” or “*Logitech VX Nano Cordless Laser Mouse*” occurs in a very small fraction of the transactions, then it can be difficult to find strong associations involving these specific items.
- Few people may buy these items together, making it unlikely that the itemset will satisfy minimum support.
- However, we would expect that it is easier to find strong associations between generalized abstractions of these items, such as between “*Dell Notebook*” and “*Cordless Mouse*.”
- Association rules generated from mining data at multiple abstraction levels are called multiple-level or multilevel association rules.

- Multilevel association rules can be mined efficiently using concept hierarchies under a support-confidence framework. In general, a top-down strategy is employed, where counts are accumulated for the calculation of frequent item sets at each concept level, starting at concept level 1 and working downward in the hierarchy toward the more specific concept levels, until no more frequent item sets can be found.
- For each level, any algorithm for discovering frequent item sets may be used, such as Apriori or its variations.
- **Using uniform minimum support for all levels** (referred to as **uniform support**): The same minimum support threshold is used when mining at each abstraction level.



- The uniform support approach, however, has some drawbacks. It is unlikely that items at lower abstraction levels will occur as frequently as those at higher abstraction levels.
- If the minimum support threshold is set too high, it could miss some meaningful associations occurring at low abstraction levels. If the threshold is set too low, it may generate many uninteresting associations occurring at high abstraction levels.
- **Using reduced minimum support at lower levels** (referred to as **reduced support**): Each abstraction level has its own minimum support threshold. The deeper the abstraction level, the smaller the corresponding threshold



- **Using item or group-based minimum support (referred to as group-based support):**
- Users or experts often have insight as to which groups are more important than others, it is sometimes more desirable to set up user-specific, item, or group-based minimal support thresholds when mining multilevel rules.
- For example, a user could set up the minimum support thresholds based on product price or on items of interest, such as by setting particularly low support thresholds for “*camera with price over \$1000*” or “*Tablet PC,*” to pay particular attention to the association patterns containing items in these categories.

- A side effect of mining multilevel association rules is its generation of many redundant rules across multiple abstraction levels due to the “ancestor” relationships among items.
- For example, consider the following rules where “*laptop computer*” is an ancestor of “*Dell laptop computer*” based on the concept hierarchy of example.
- *X* is a variable representing customers who purchased items in *AllElectronics* transactions.

$$\text{buys}(X, \text{“laptop computer”}) \Rightarrow \text{buys}(X, \text{“HP printer”})$$
$$[\text{support} = 8\%, \text{confidence} = 70\%]$$

$$\text{buys}(X, \text{“Dell laptop computer”}) \Rightarrow \text{buys}(X, \text{“HP printer”})$$
$$[\text{support} = 2\%, \text{confidence} = 72\%]$$

- *If both the Rules are mined, then how useful is Rule (2)? Does it really provide any novel information?* If the latter, less general rule does not provide new information, then it should be removed.
- A rule $R1$ is an **ancestor** of a rule $R2$, if $R1$ can be obtained by replacing the items in $R2$ by their ancestors in a concept hierarchy.
- In this example “*laptop computer*” is an ancestor of “*Dell laptop computer*.” Based on this definition, a rule can be considered redundant if its support and confidence are close to their “expected” values, based on an ancestor of the rule.

Multi-Level Mining: Progressive Deepening

- A top-down, progressive deepening approach:
 - First mine high-level frequent items:
milk (15%), bread (10%)
 - Then mine their lower-level “weaker” frequent item sets:
2% milk (5%), wheat bread (4%)
- Different min_support threshold across multi-levels lead to different algorithms:
 - If adopting the same min_support across multi-levels then toss t if any of t 's ancestors is infrequent.
 - If adopting reduced min_support at lower levels then examine only those descendents whose ancestor's support is frequent/non-negligible.

Progressive Refinement of Data Mining Quality

- **Why progressive refinement?**
 - Mining operator can be expensive or cheap, fine or rough
 - Trade speed with quality: step-by-step refinement.
- **Superset coverage property:**
 - Preserve all the positive answers—allow a positive false test but not a false negative test.
- **Two- or multi-step mining:**
 - First apply rough/cheap operator (superset coverage)
 - Then apply expensive algorithm on a substantially reduced candidate set (Koperski & Han, [SSD'95](#)).

Multi-Dimensional Association: Concepts

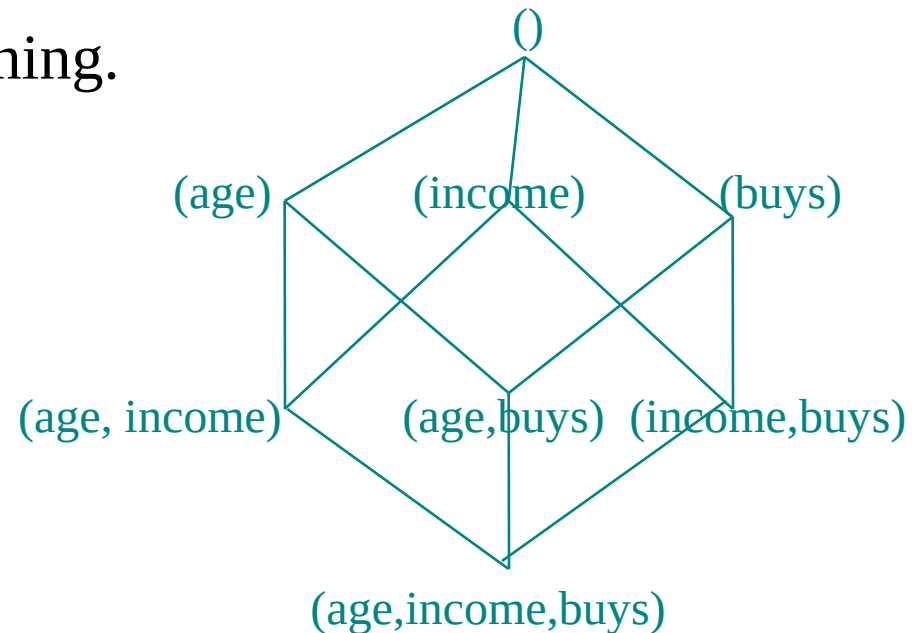
- Single-dimensional rules:
 $\text{buys}(X, \text{"milk"}) \Rightarrow \text{buys}(X, \text{"bread"})$
- Multi-dimensional rules: $\square$ 2 dimensions or predicates
 - Inter-dimension association rules (*no repeated predicates*)
 $\text{age}(X, \text{"19-25"}) \wedge \text{occupation}(X, \text{"student"}) \Rightarrow \text{buys}(X, \text{"coke"})$
 - hybrid-dimension association rules (*repeated predicates*)
 $\text{age}(X, \text{"19-25"}) \wedge \text{buys}(X, \text{"popcorn"}) \Rightarrow \text{buys}(X, \text{"coke"})$
- Categorical Attributes
 - finite number of possible values, no ordering among values
- Quantitative Attributes
 - numeric, implicit ordering among values

Techniques for Mining MD Associations

- Search for frequent k -predicate set:
 - Example: {age, occupation, buys} is a 3-predicate set.
 - Techniques can be categorized by how age is treated:
- 1. Using static discretization of quantitative attributes
 - Quantitative attributes are statically discretized by using predefined concept hierarchies.
- 2. Quantitative association rules
 - Quantitative attributes are dynamically discretized into “bins” based on the distribution of the data.
- 3. Distance-based association rules
 - This is a dynamic discretization process that considers the distance between data points.

Static Discretization of Quantitative Attributes

- Discretized prior to mining using concept hierarchy.
- Numeric values are replaced by ranges.
- In relational database, finding all frequent k -predicate sets will require k or $k+1$ table scans.
- Data cube is well suited for mining.
- The cells of an n -dimensional cuboid correspond to the predicate sets.
- Mining from data cubes can be much faster.



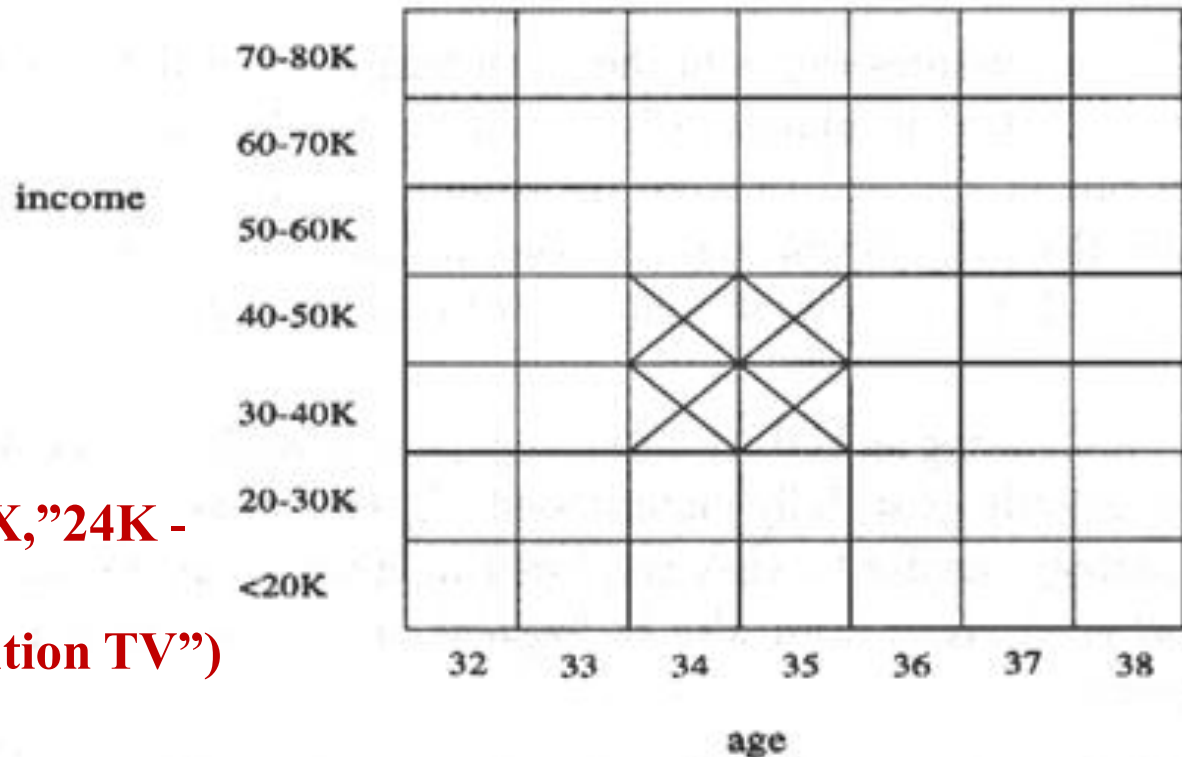
Quantitative Association Rules

- Numeric attributes are *dynamically* discretized
 - Such that the confidence or compactness of the rules mined is maximized.
- 2-D quantitative association rules: $A_{\text{quan1}} \wedge A_{\text{quan2}} \Rightarrow A_{\text{cat}}$
- Cluster “adjacent”

association rules
to form general
rules using a 2-D
grid.

- Example:

$\text{age}(X, "30-34") \wedge \text{income}(X, "24K - 48K")$
 $\Rightarrow \text{buys}(X, "high\ resolution\ TV")$



ARCS (Association Rule Clustering System)

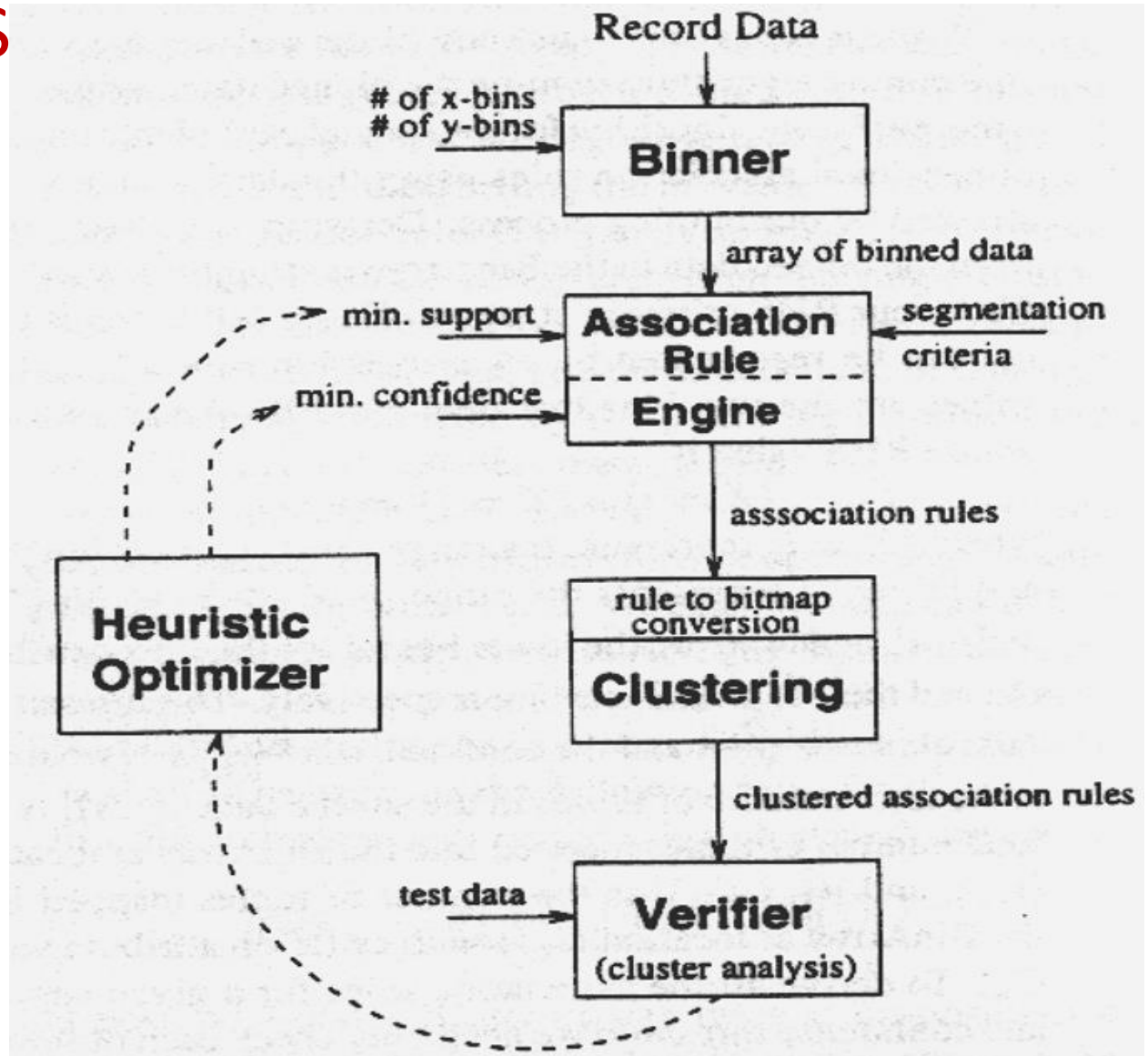
How does ARCS work?

1. Binning

2. Find frequent predicate set

3. Clustering

4. Optimize



Limitations of ARCS

- Only quantitative attributes on LHS of rules.
- Only 2 attributes on LHS. (2D limitation)
- An alternative to ARCS
 - Non-grid-based
 - equi-depth binning
 - clustering based on a measure of *partial completeness* (*information lost due to partitioning*).
 - “***Mining Quantitative Association Rules in Large Relational Tables***” by R. Srikant and R. Agrawal.

From Association to Correlation Analysis

- **A misleading “strong” association rule.** Suppose we are interested in analysing transactions with respect to the purchase of computer games and videos.
- Let *game* refer to the transactions containing computer games, and *video* refer to those containing videos. Of the 10,000 transactions analyzed, the data show that 6000 of the customer transactions included computer games, while 7500 included videos, and 4000 included both computer games and videos.
- Suppose that a data mining program for discovering association rules is run on the data, using a minimum support of, say, 30% and a minimum confidence of 60%.
- The following association rule is discovered:

$$\text{buys}(X, \text{“computer games”}) \Rightarrow \text{buys}(X, \text{“videos”})$$
$$[\text{support} = 40\%, \text{confidence} = 66\%].$$

- Rule is a strong association rule and would therefore be reported, since its support value of $4000/10000 = 40\%$ and confidence value of $4000/6000 = 66\%$ satisfy the minimum support and minimum confidence thresholds, respectively.
- However, Rule is misleading because the probability of purchasing videos is 75%, which is even larger than 66%.
- In fact, computer games and videos are negatively associated because the purchase of one of these items actually decreases the likelihood of purchasing the other.
- Without fully understanding this phenomenon, we could easily make unwise business decisions based on Rule.

From Association to Correlation Analysis

- ▶ Use **Lift**, a simple correlation measure
- ▶ The occurrence of itemset **A** is independent of the occurrence of itemset **B** if $P(A \cup B) = P(A)P(B)$, otherwise itemsets **A** and **B** are dependent and correlated as events
- ▶ The lift between the occurrences of **A** and **B** is given by

$$\text{Lift}(A, B) = P(A \cup B) / P(A)P(B)$$

- If > 1 , then A and B are positively correlated (the occurrence of one implies the occurrence of the other)
- If < 1 , then A and B are negatively correlated
- If $= 1$, then A and B are independent
- ▶ **Example:** $P(\{\text{game}, \text{video}\}) = 0.4 / (0.60 \times 0.75) = 0.89$

Subjective Arguments

- A pattern is considered subjectively uninteresting unless it reveals unexpected information about the data.
- E.g., the rule {Butter} → {Bread} isn't interesting, despite having high support and confidence values.
- On the other hand, the rule {Diapers} → {Beer} is interesting because the relationship is quite unexpected and may suggest a new crossselling opportunity for retailers.
- **Drawback:** Incorporating subjective knowledge into pattern evaluation is a difficult task because it requires a considerable amount of prior information from the domain experts.

Computing Interestingness Measures

- Given a rule $X \rightarrow Y$, the information needed to compute rule interestingness can be obtained from a contingency table

Contingency table for $X \rightarrow Y$

	Y	$\bar{Y}$	
X	f_{11}	f_{10}	f_{1+}
$\bar{X}$	f_{01}	f_{00}	f_{0+}
	f_{+1}	f_{+0}	$ T $

f_{11} : support of X and Y

f_{10} : support of $\underline{X}$ and $\bar{Y}$

f_{01} : support of $\bar{X}$ and $\underline{Y}$

f_{00} : support of $\bar{X}$ and $\bar{Y}$

Used to define various measures

Pitfall of Confidence

	Coffee	¬Coffee	
Tea	150	50	200
¬Tea	750	150	900
	900	200	1100

The pitfall of confidence can be traced to the fact that the measure ignores the support of the itemset in the rule consequent.

Consider **association rule**: Tea → Coffee

Confidence=

$$P(\text{Coffee}, \text{Tea}) / P(\text{Tea}) = P(\text{Coffee} | \text{Tea}) = 150 / 200 = 0.75 \text{ (seems quite high)}$$

But, $P(\text{Coffee}) = 0.9$

Thus knowing that a person is a tea drinker actually decreases his/her probability of being a coffee drinker from **90%** to **75%**!

⇒ Although confidence is high, rule is misleading

In fact $P(\text{Coffee} | \neg \text{Tea}) =$

$$P(\text{Coffee}, \neg \text{Tea}) / P(\neg \text{Tea}) = 750 / 900 = 0.83$$

Statistical Independence

- Population of 1000 students
- 600 students know how to swim (S)
- 700 students know how to bike (B)
- 420 students know how to swim and bike (S,B)
- $P(S|B) = P(S)$ ($P(S \cap B)/P(B) = .42 / .7 = .6 = P(S)$)
- $P(S \cap B)/P(B) = P(S)$
- $P(S \cap B) = P(S) \times P(B) \Rightarrow$ **Statistical independence**
- $P(S \cap B) > P(S) \times P(B) \Rightarrow$ **Positively correlated**
 - i.e. if someone knows how to swim, then it is more probable he knows how to bike, and vice versa
- $P(S \cap B) < P(S) \times P(B) \Rightarrow$ **Negatively correlated**
 - i.e. if someone knows how to swim, then it is less probable he/she knows how to bike, and vice versa

Interest Factor

- Measure that takes into account statistical dependence

$$Interest = \frac{P(A \wedge B)}{P(A)P(B)} = \frac{f_{11}/N}{(f_{1+}/N) \times (f_{+1}/N)} = \frac{N \times f_{11}}{f_{1+} \times f_{+1}}$$

- Interest factor compares the frequency of a pattern against a baseline frequency computed under the statistical independence assumption.
- The **baseline** frequency for a pair of mutually independent variables is:

$$\frac{f_{11}}{N} = \frac{f_{1+}}{N} \times \frac{f_{+1}}{N} \quad \text{Or equivalently} \quad f_{11} = \frac{f_{1+} \times f_{+1}}{N}$$

Interest Equation

- Fraction f_{11}/N is an estimate for the joint probability $P(A,B)$, while f_{1+}/N and f_{+1}/N are the estimates for $P(A)$ and $P(B)$, respectively.
- If A and B are statistically independent, then $P(A \wedge B) = P(A) \times P(B)$, thus the **Interest is 1**.

$$I(A, B) \begin{cases} = 1, & \text{if } A \text{ and } B \text{ are independent;} \\ > 1, & \text{if } A \text{ and } B \text{ are positively correlated;} \\ < 1, & \text{if } A \text{ and } B \text{ are negatively correlated.} \end{cases}$$

Example: Interest

	Coffee	¬Coffee	
Tea	150	50	200
¬Tea	750	150	900
	900	200	1100

Association Rule: Tea $\rightarrow$ Coffee

Interest =

$$150 * 1100 / (200 * 900) = 0.92$$

(< 1, therefore they are negatively correlated)